Game play --- An great AI Triumph

Chess play is considered the best example of human intelligence.

1997, IBM Deep Blue defeated International Chess world champion.

2016, Google AlphaGo defeated Go world champion.

AI comes to life.

Here we explain chess play technology use: Tic-Tac-Toe

Chris Ding

国际跳棋Checkers





Primary school

- **In 1959, IBM engineer Samuel made a computer beat ordinary chess player**
- **In 1967, IBM computer beat state champion**
- **1994 University of Alberta beat world checker champion**

International Chess







1997 Deep Blue defeated World Champion Kasparov



AI Great Achievement by using search related algorithms

1997 Deep Blue defeated World Champion Kasparov



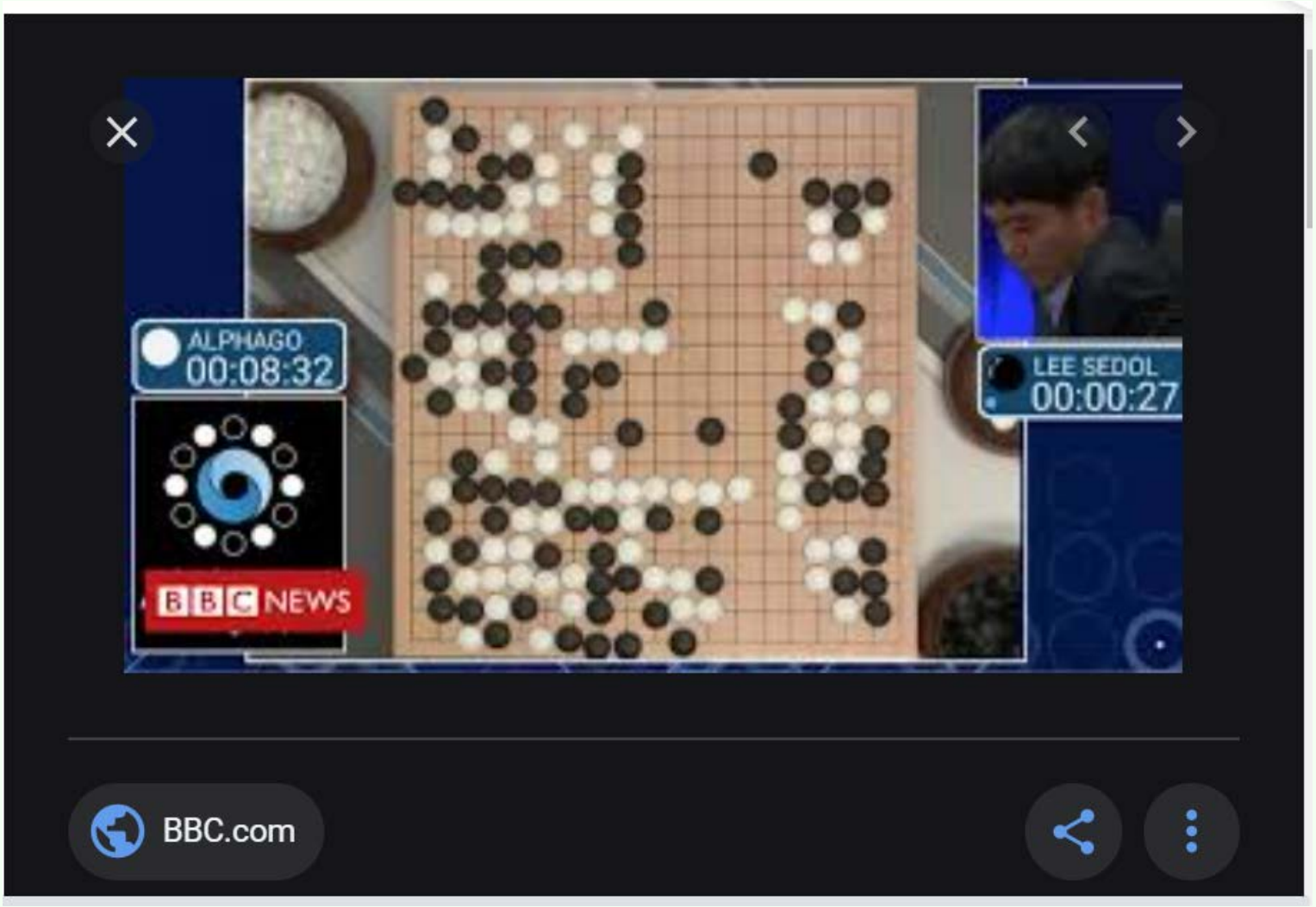
NEW YORK, UNITED STATES: The IBM Deep Blue chess computer team poses in the playing room 08 May 1997 in New York. From L are: Chung-Jen Tan (team manager), Gerry Brody, Joel Benjamin, Murray Campbell, Joseph Hoane and Feng-Hsiung Hsu (seated).



go



2016 Google AlphaGo beats *World Go Champion* Lee Sedol



International Chess has





The key technology of game play is search.

International chess search space $O(10^{11})$



Go search space $O(10^{170})$



From 1997 IBM Blue to 2016 Google AlphaGo, humans spent 20 years to develop search technology to make

100000000000000000000 …… 0000000000000000 (160 zeros)

folds leap to conquer the Go.

in the process, they won Nobel prize ……

Int'l Chess

- 8 x 8 board
- Each player has 16 pieces (棋子), such as king(帅), knight(马), bishop(象相), rook(车)
- Each piece () has strict moves and all different moves by all pieces are around 38 (branching ratio in search tree)
- Search space $O(10^{11})$

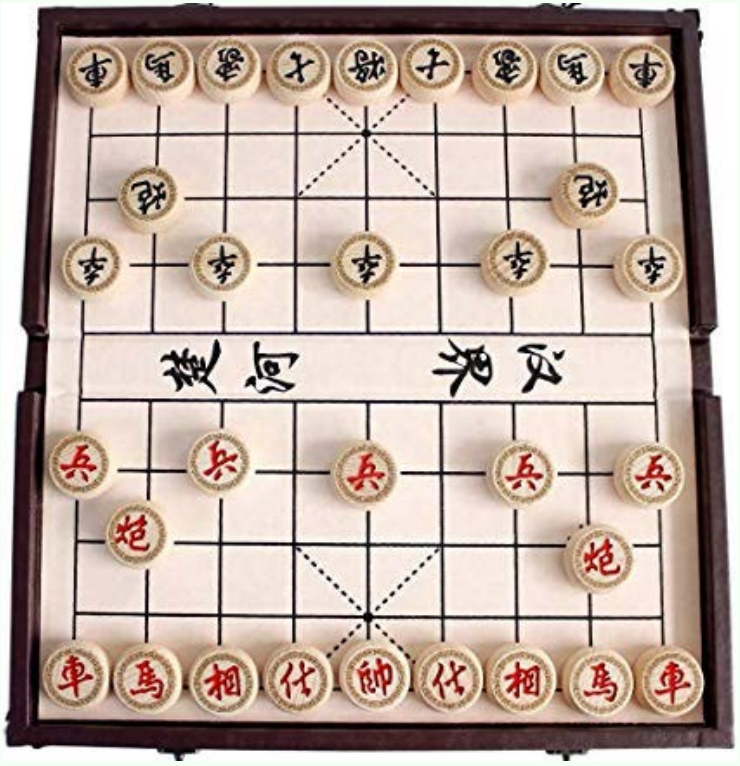


Go

- 19x19 board.
- You can put a piece in all spaces (19x19=361 to about half of that) Average branching ratio ≈ 200 in search tree.
- Search space $O(10^{170})$



Chinese chess





棋类游戏，或者更具体的来说，国际象棋或围棋，通常被认为是人类智慧的最佳例证。

1997年，IBM的深蓝计算机击败了国际象棋世界冠军。
2017年，谷歌的阿尔法狗击败了世界围棋冠军。
人工智能时代到来。

2024年，从下棋技术AlphaZero发展出来AlphaFold荣获诺贝尔化学奖

相比卷积神经网络 或 ChatGPT语言大模型，人工智能下棋技术，简单很多，能下一两步象棋的人，就能理解其中的关键技术。

下象棋，熟手能看两三步棋，高手能算五六步棋，计算机能算十几步棋。

中心技术:从一个棋局，搜索下一步棋局，再搜索下下一步棋局，等等，整个思路清晰。

所以1950年代末期就开始研究。

A list of big numbers

If you write $10^{100000000}$, it has zero meaning!

A book has about 10^{22}atoms

A human has 36 trillion $36x10^{12}$ cells

A person walks from earth to moon take 548 million $5.48x10^{6}$ steps

Open AI company market value is 500 Billion $5x10^{11}$ USD (CEO Sam Altman wants to raise 7 trillion for general AI. US GDP is 25 trillion, China GDP is 18 trillion)

Search engine google name comes from googol = 1 followed with 100 zeros (10^{100})

Search space of International Chess = 10^{11} diffrents moves

Search space of chess Go = 10^{170} different moves

Giant steps on search technology developments

From **1997** when IBM Deep Blue computer beats **World Chess** Champion Kasparov, to **2016** when Google AlphaGo beats **Go** World Champion, huamns took 20 years to develop more efficient search algorithms from searching O(10^{11}) space of *International Chess* to searching O(10^{170}) space of *GO*.

This is about walking to moon and come back round trip about 10^{159} times!

从1997年征服国际象棋，到2017年征服围棋，人类用了20年时间 发展**搜索技术**，来解决 搜索空间从10^{11} 到 10^{170} 登天级的跳跃。

https://www.chessdb.cn/query/

中国象棋云库查询

输入FEN
复制FEN
设置URL
初始局面
空白局面
翻转棋盘

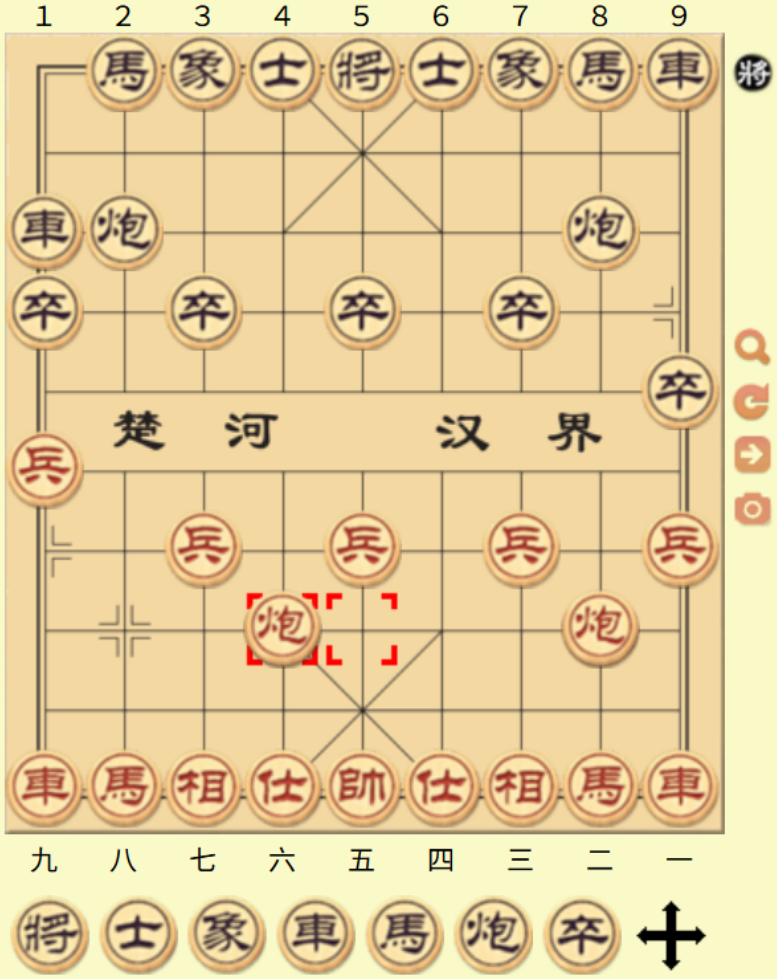
◉ 黑方走棋
○ 红方走棋

☑ 棋规裁定
☐ 隐藏结果

☐ 自动走黑
☐ 自动走红

自动走棋策略
◉ 最佳着法
○ 合理着法
☑ 在线计算

残局库类型
○ DTC
◉ DTM



着法	排序	分数	备注
炮２平５	2	16	! (44-02)
炮８平５	1	7	* (45-01)
卒３进１	0	2	? (44-02)
卒７进１	0	-3	? (44-03)
马８进９	0	-15	? (43-01)
炮８平６	0	-18	? (45-03)
马８进７	0	-24	? (43-03)
炮２进４	0	-31	? (44-01)
象７进５	0	-40	? (44-02)
士４进５	0	-43	? (45-01)
炮８平７	0	-45	? (45-01)
炮２平４	0	-45	? (43-01)
炮８平９	0	-46	? (45-01)
炮２平３	0	-46	? (44-02)
炮２退１	0	-47	? (44-01)
炮２平６	0	-58	? (44-01)
卒９进１	0	-59	? (44-01)
炮８平４	0	-62	? (44-01)
炮２平７	0	-63	? (44-01)
士６进５	0	-65	? (44-01)
马２进３	0	-69	? (44-01)
炮８平３	0	-80	? (45-01)
车１退１	0	-81	? (44-01)
象３进５	0	-84	? (44-01)

谁会想到计算机能打败世界冠军，获得诺贝尔奖！

Chess Player vs. Computer Game

The Simplest Strategy: The Computer Looks 1-step Ahead

- **Player's Move:** The human player makes a move.
- **Computer's Move:** From the current board state, the computer searches all possible next moves. It computes an **heuristic evalution score** to each resulting board state and selects the move with the highest score.

Chess Player vs. Computer Game

Strategy where the Computer Looks 3-step Ahead

•**Player** makes a 1st move. The current board state is now A1.

•**Computer's Move:**

- From board state A1, the computer searches all possible next moves it can make, resulting in board states B1, B2, B3, etc.
- **Simulate Player's next move**: For each of the computer's possible moves (B1, B2, B3...), the computer simulates all possible counter-moves the player could make, resulting in a new set of board states (A2, A2', A2''... for B1; A2, A2', A2''... for B2; and so on).
- **Simulate Computer's 2nd move**: For each of the player's simulated moves (A2, A2', A2''...), the computer then searches for its own next possible moves, resulting in board states B2, B2', B2''..., and **scores** them.
- Using the **minimax algorithm**, these scores are propagated back up the tree. The computer assumes the player will choose moves that minimize the computer's advantage.
- Finally, the computer selects the move from the first set (B1, B2, B3...) that leads to the best possible outcome after this simulated sequence of moves.

The chess board state evaluation/heuristic function is extremely important. The effectiveness of looking many moves ahead depends entirely on it.

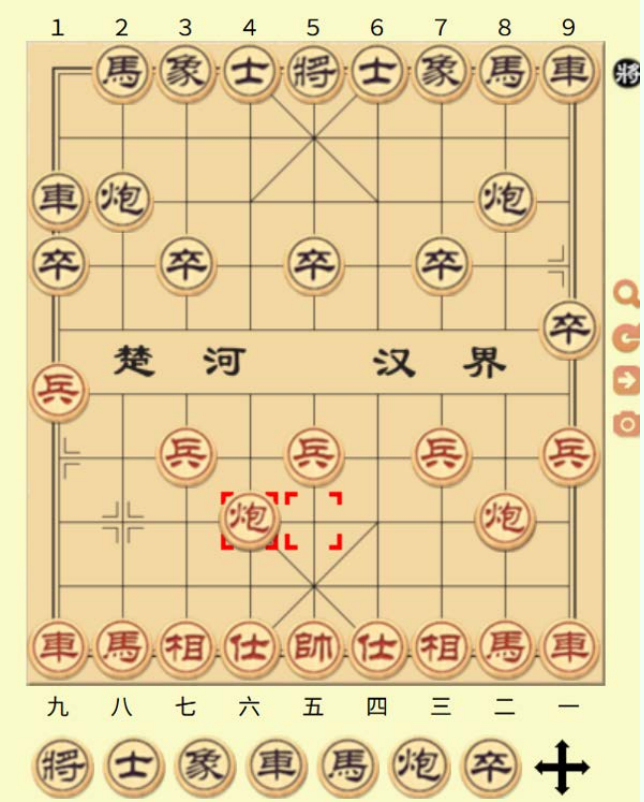
Homework-Game1: branching factor in games.

Suppose it is now ”red” player’s turn. See board situation on left figure.

(A) How many different/distinct moves he can make? Write down details. For instance,
5个 兵(solider) has 5 moves
Left 炮 (cannon) has 3+3+1+6=13 moves
…
This value is the branching factor, or number of adjacent nodes on the chess board situation graph.

(B) If he use 1-step look-ahead strategy, how many evaluation of heuristic (assessment of the board situation) evaluation he needs to do?

(C) If he use 3-step look-ahead strategy, how many evaluation of heuristic evaluation he needs to do?



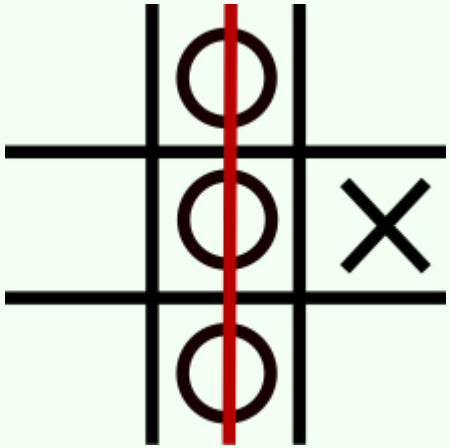
Tic Tac Toe Game

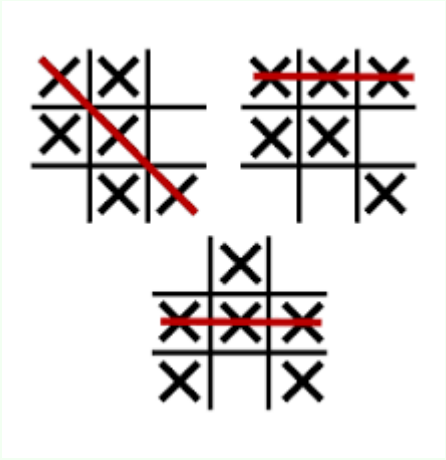












Outline

We first play several Tic-Tac-Toe games to become familiar with the situation.

We introduce the basic algorithm: BFS. For each player to make a move, the computer searches all the way to the end of the game to determine the most move. This introduces the minimax search.

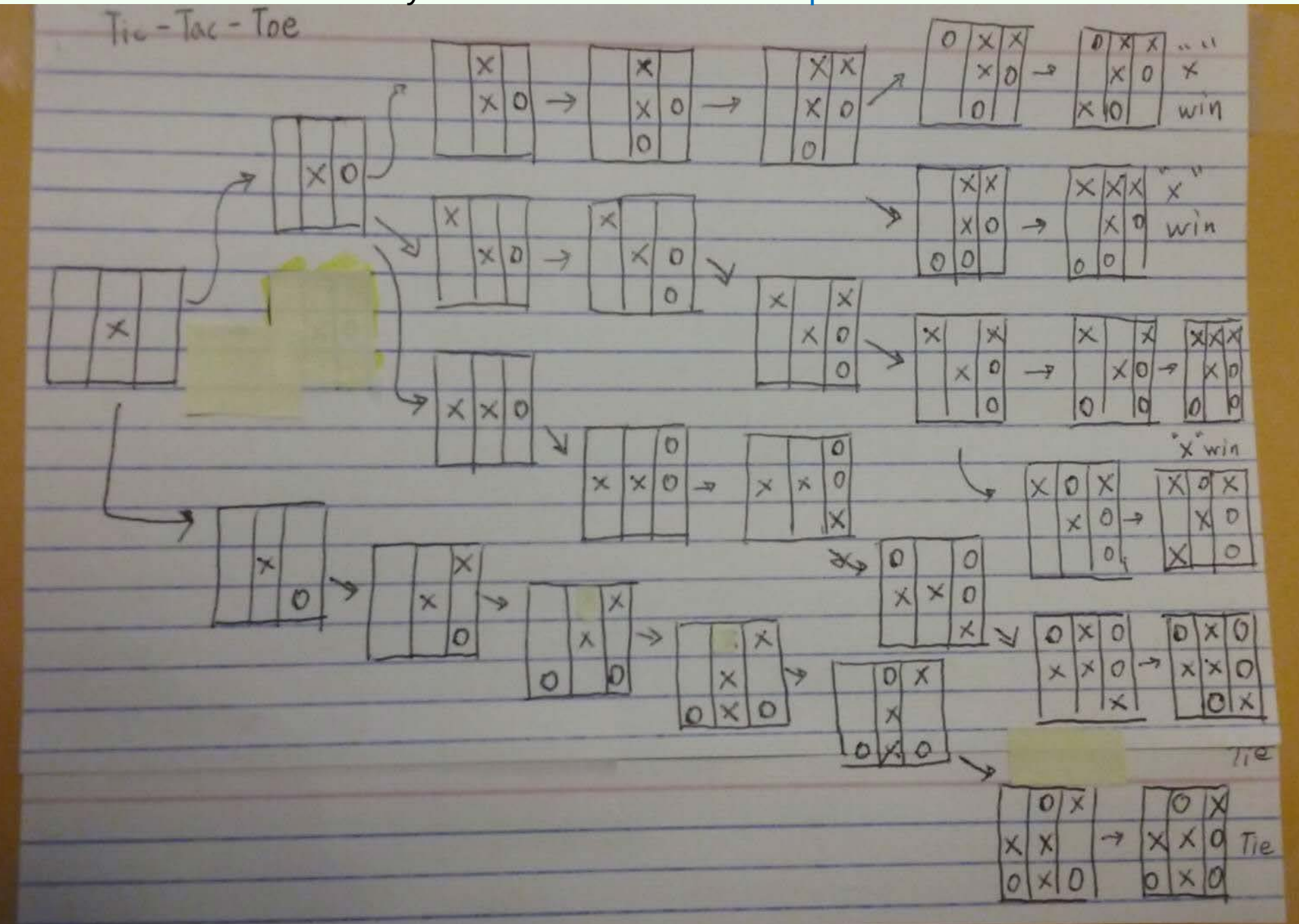
We then introduce the improved algorithm: Depth-limited (e.g. 3-step) DFS. This requires a heuristic score. Backtrack: valuebackup

We end up with the fast algorithm: Minimax Search

(All these require good understanding of BSF, DFS algorithms)

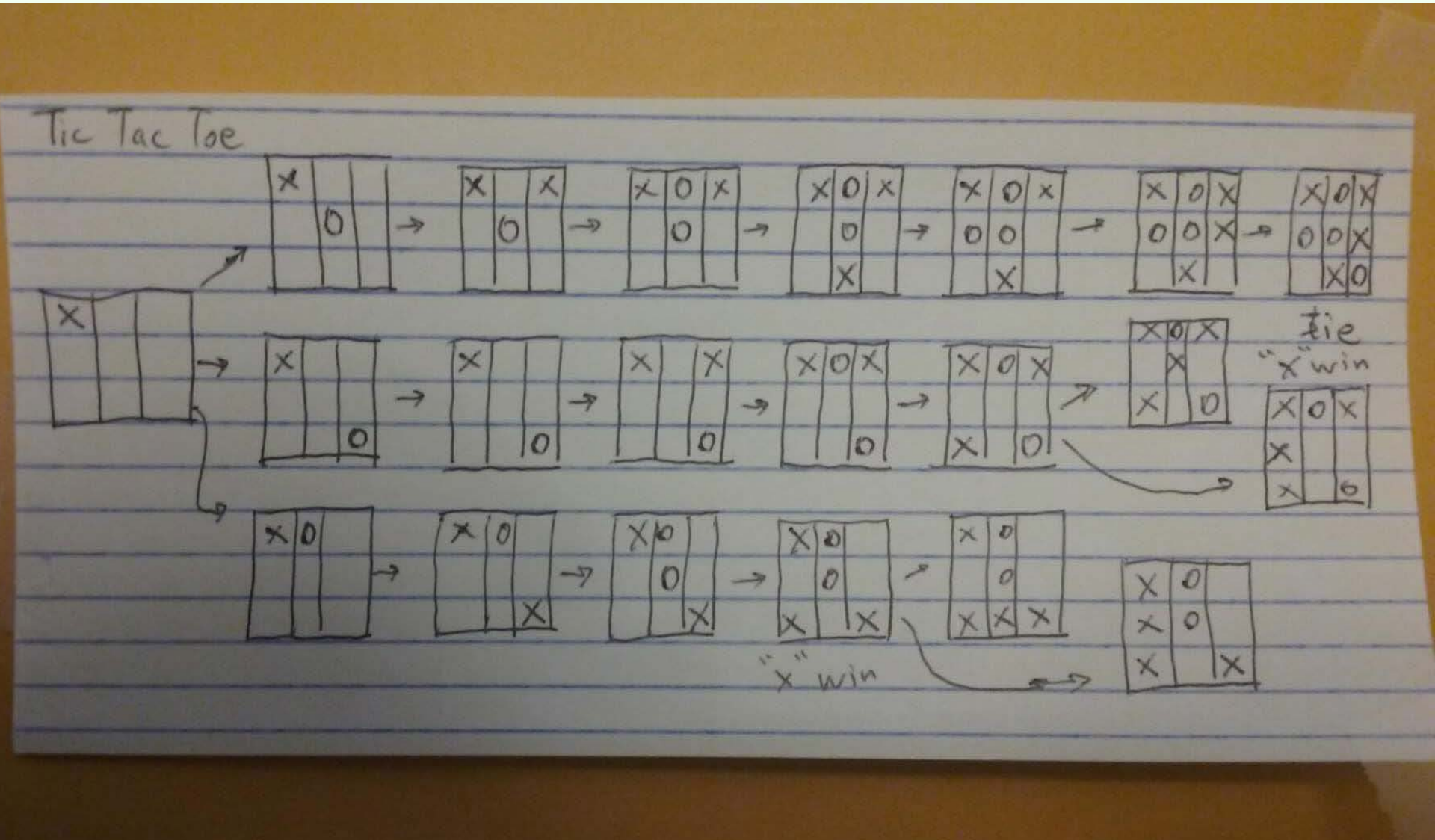
We first play the game several times
to get familiar with the problem.

Play Tic-Tac-Toe: several examples

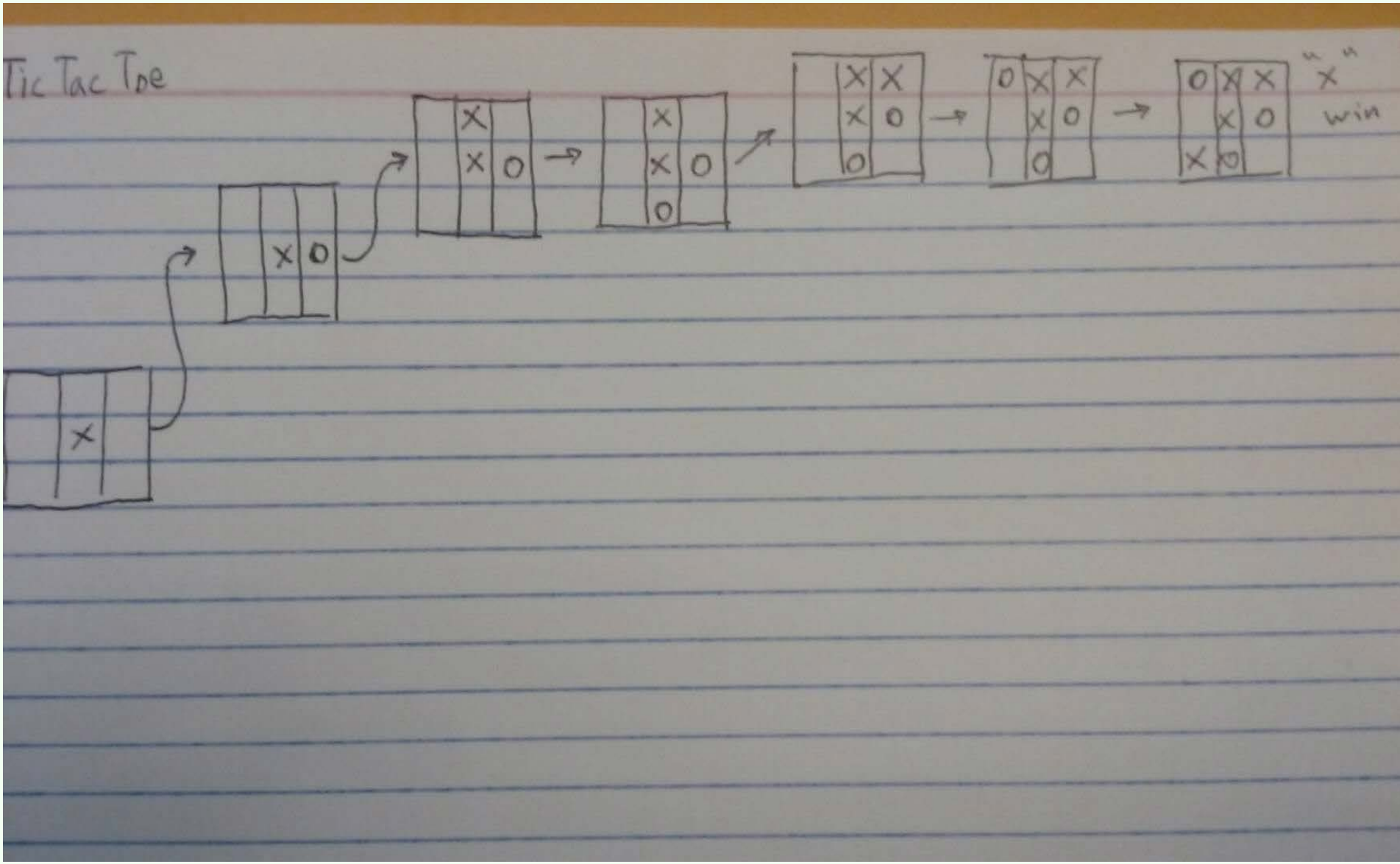


“x” is the first player to play. We call this player “max”. Later we will see that *max* wants to make a move (put in one checker) that maximize the heurisic score.

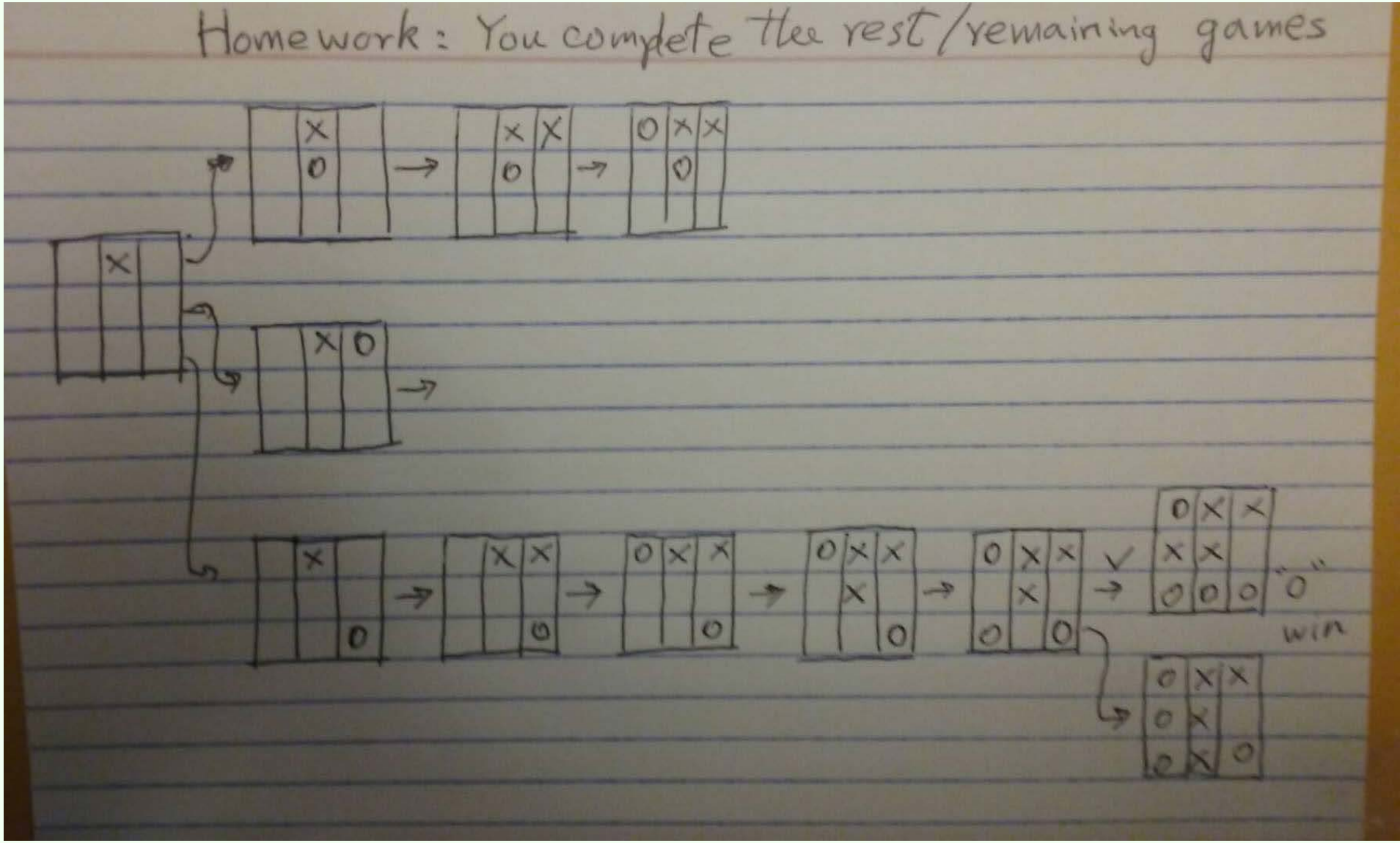
“o” is the second player to play. We call this player “min”. Later we will see that *min* wants to make a move (put in one checker) that minimize the heurisic score.



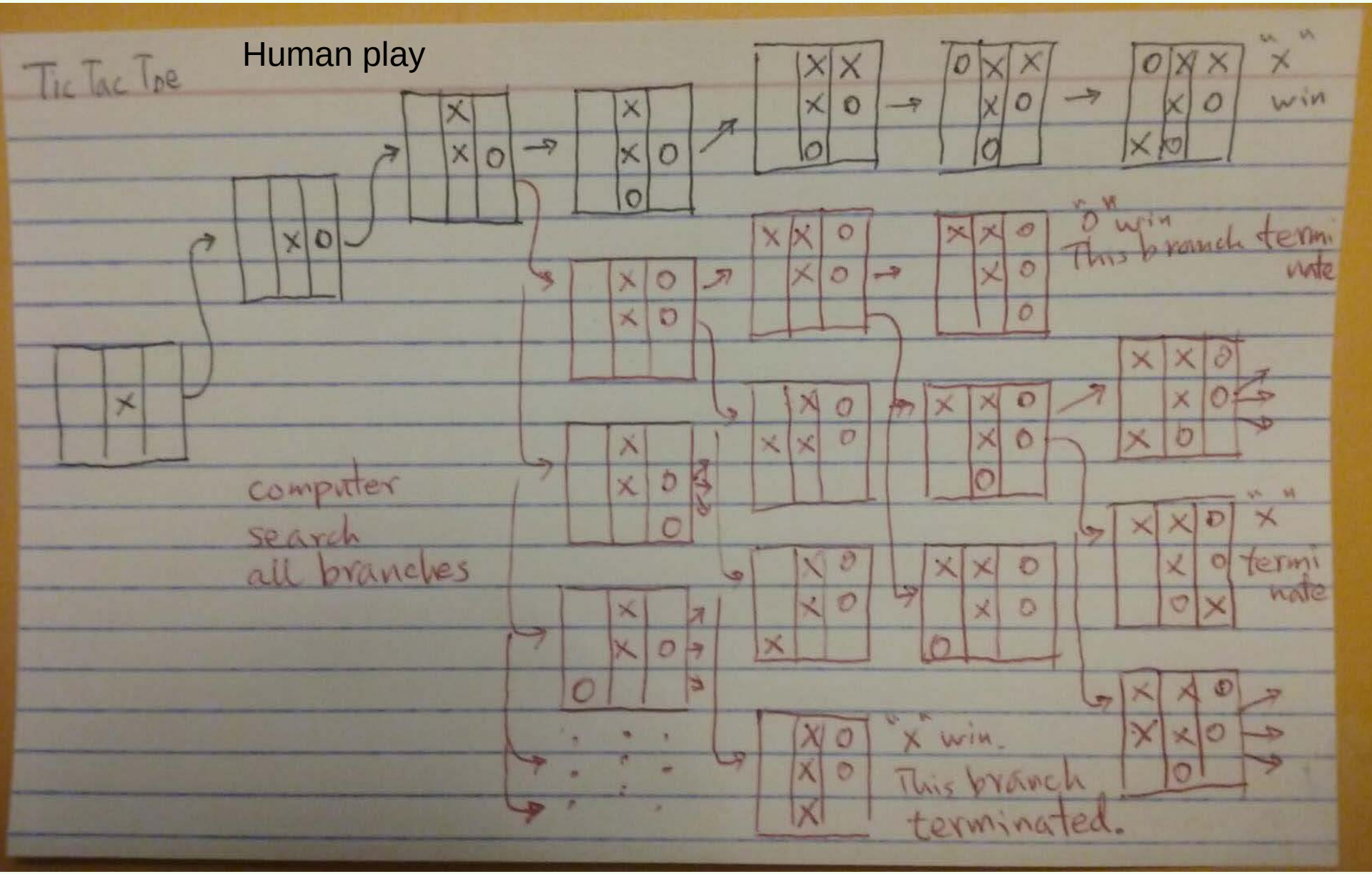
Tic-Tac-Toe DFS Search Tree



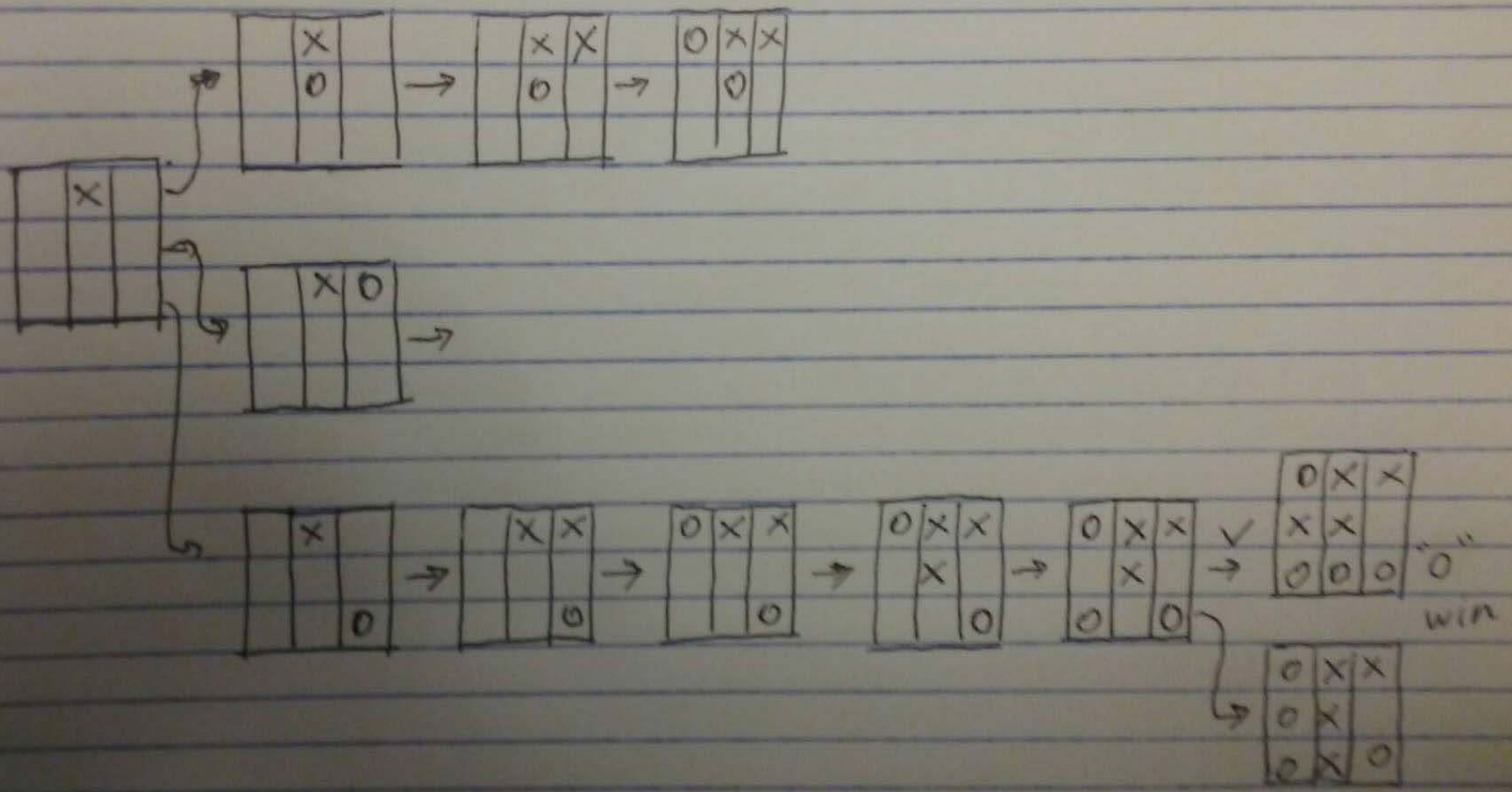
Homework-Game2: Complete the rest steps of the top two incomplete game plays using your own instincts.



Tic-Tac-Toe BFS Search Tree



Homework : You complete the rest / remaining games



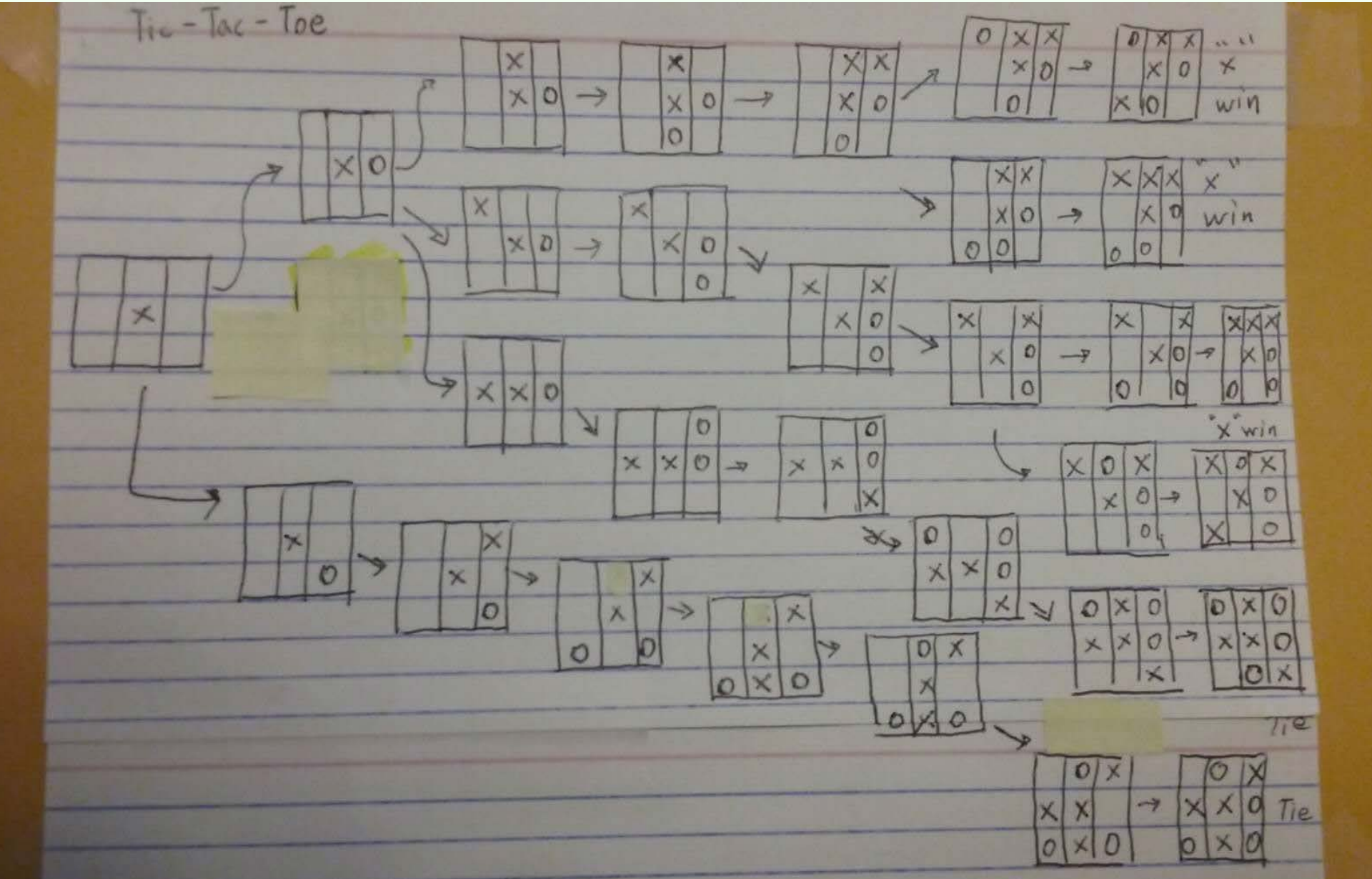
How to solve the Tic-Tac-Toe using a computer?

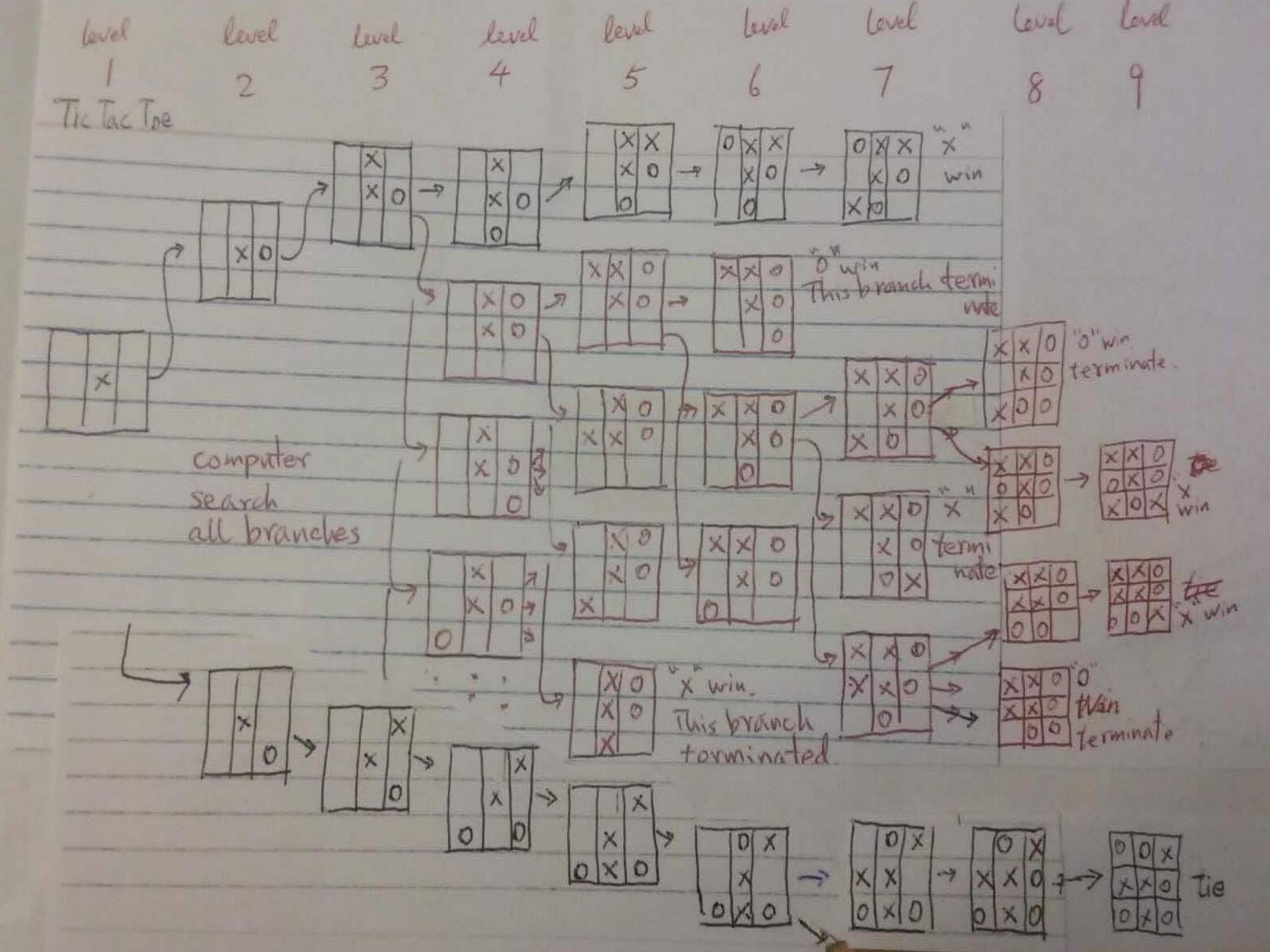
Basic algorithm:

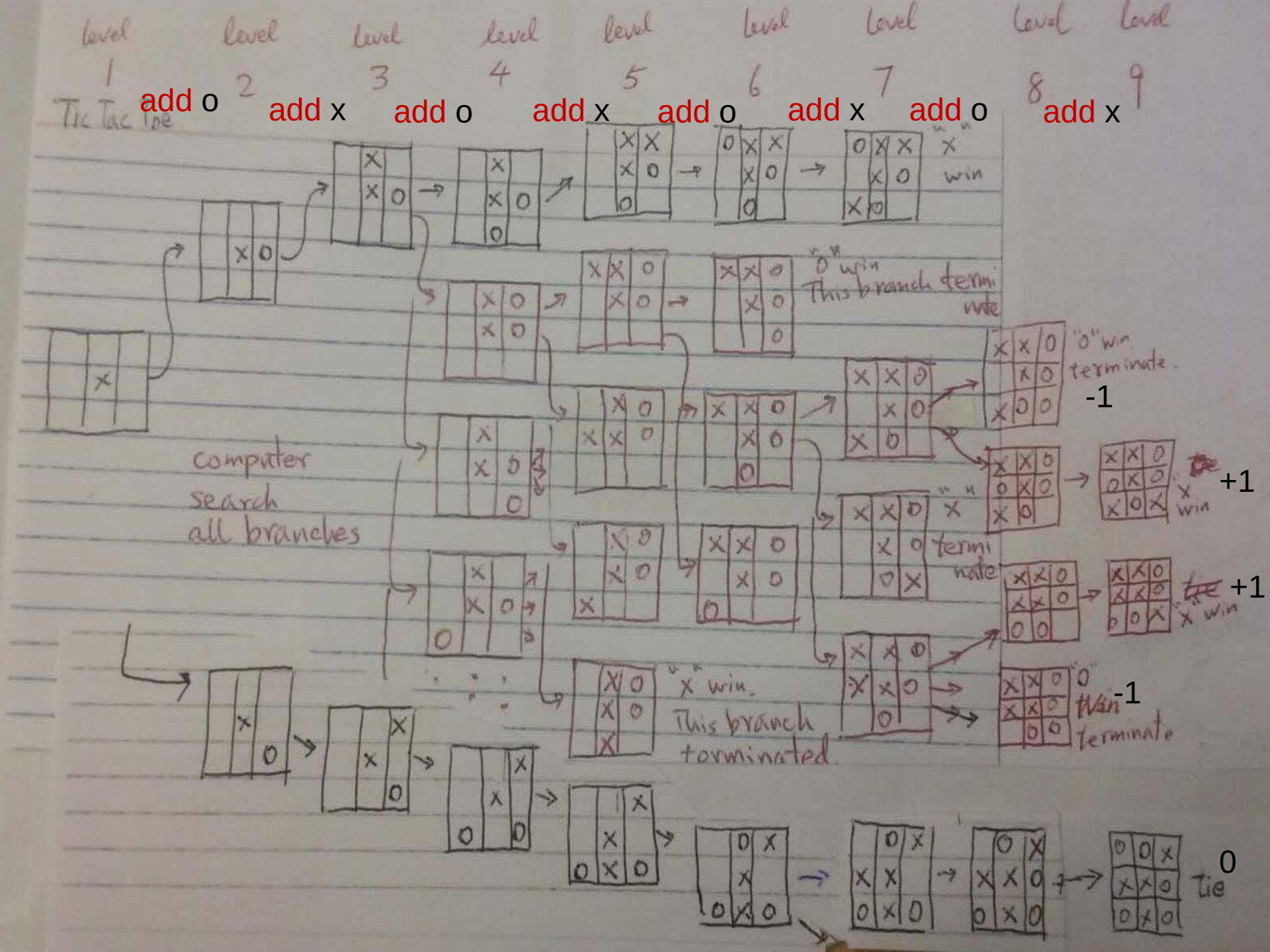
At a node/state/configuration, we need to make a move.
Computer search all possible moves at this stage to get a score (play all the way to the end to get a win/lose/tie). Pick the move with the best score.

This may requires too large space to store intermediate results.

Suppose 1st player already put x at the center. 2nd player can put o in 2 places. Computer plays all-the-way to the end. This forms a tree. Each leaf node is (+1, -1, 0), (+1=win for x , -1=win for o, 0=tie).




level 1
level 2
level 3
level 4
level 5
level 6
level 7
level 8
level 9
Tic Tac Toe
"X" win
"O" win
This branch terminate
"O" win terminate.
computer search all branches
terminate
X win
tie
"X" win.
This branch terminated.
"O" win terminate
tie


level 1
level 2
level 3
level 4
level 5
level 6
level 7
level 8
level 9
Tic Tac Toe
add o
add x
add o
add x
add o
add x
add o
add x
"x" win
"o" win
This branch terminate
"o" win terminate.
-1
computer search all branches
+1
terminate
+1
"x" win.
This branch terminated.
-1
terminate
tie
0

Value Backup

At bottom level, level 9, do value backup. Since this level is “add x”, i.e., max’s move, he picks maximum scores his children at level 9 to fill the scores at level 8.
(Question: at level 9, only scores +1 or 0 at the leafs. Why?)

A subtree starting from a situation/configuration/state

Value Backup. **Scores are shown for each situation**

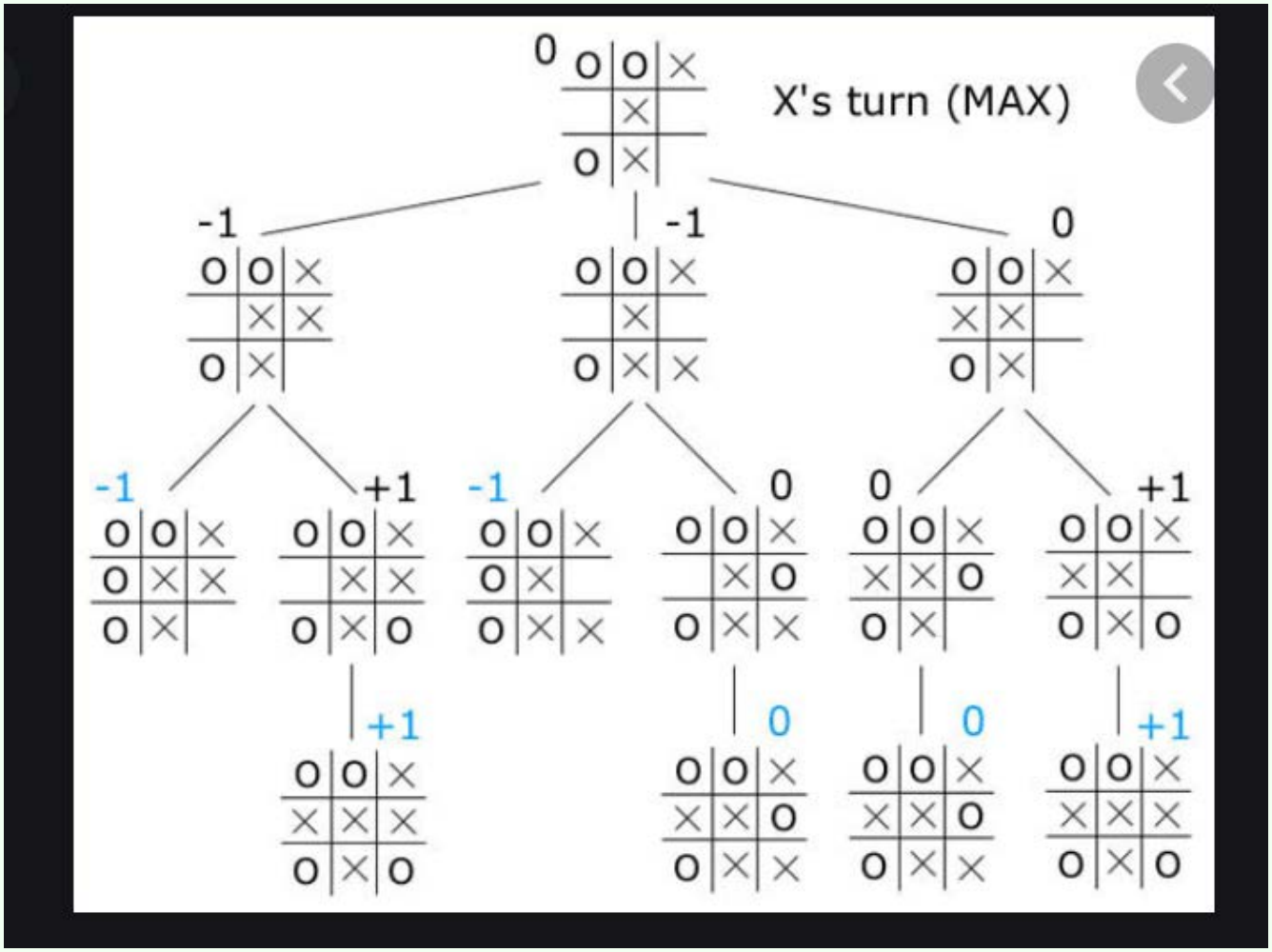
level 7 max up

min up

level 8

max up

level 9



Homework: 1. Identify the leaf nodes in this subtree.

2. Show the tree if we do minimax search algorithm from the starting node.

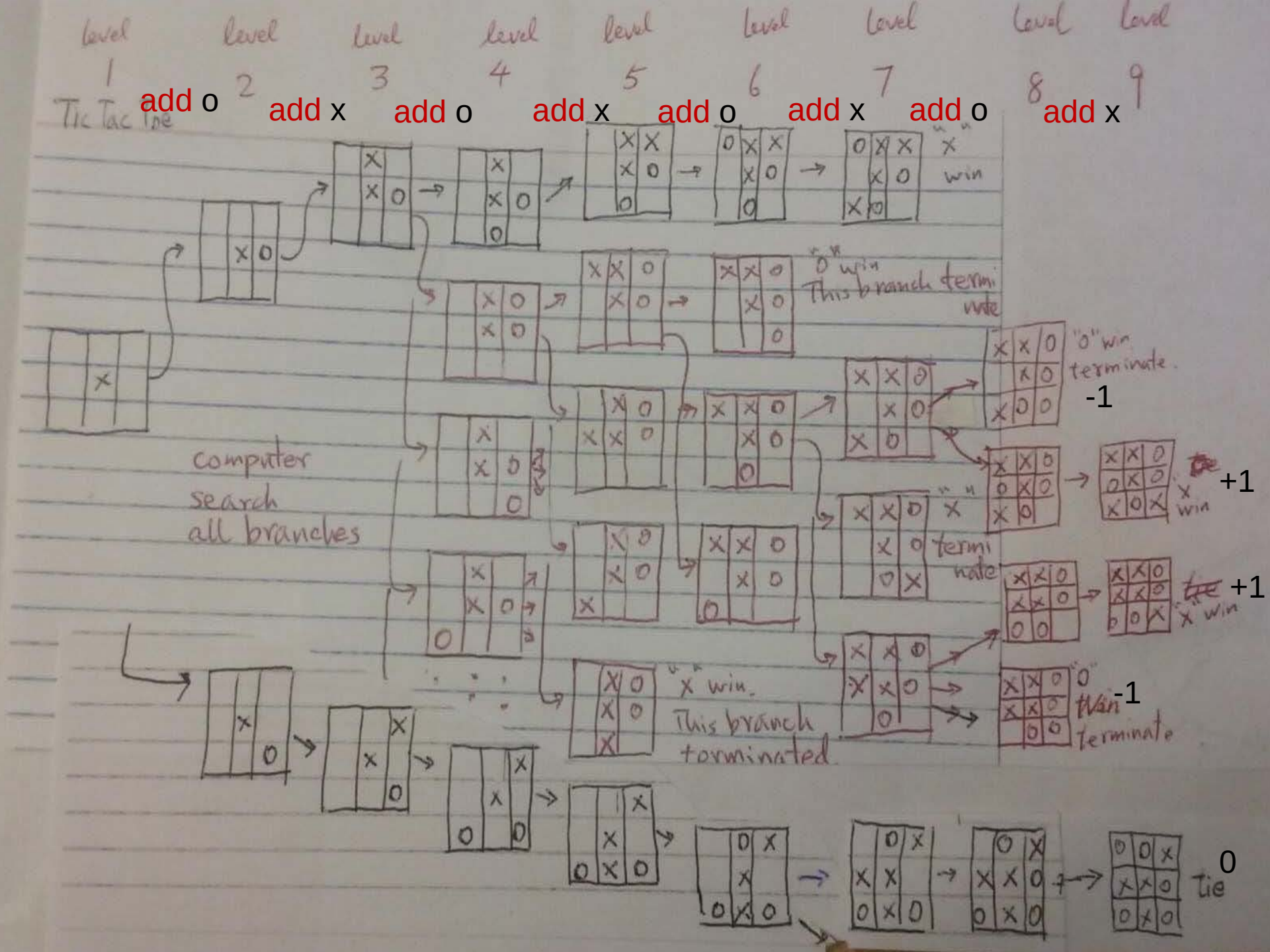
Value Backup

At bottom level, level 9, do value backup. Since this level is “add x”, i.e., max’s move, he picks maximum scores his children at level 9 to fill the scores at level 8.
(Question: at level 9, only scores +1 or 0 at the leafs. Why?)

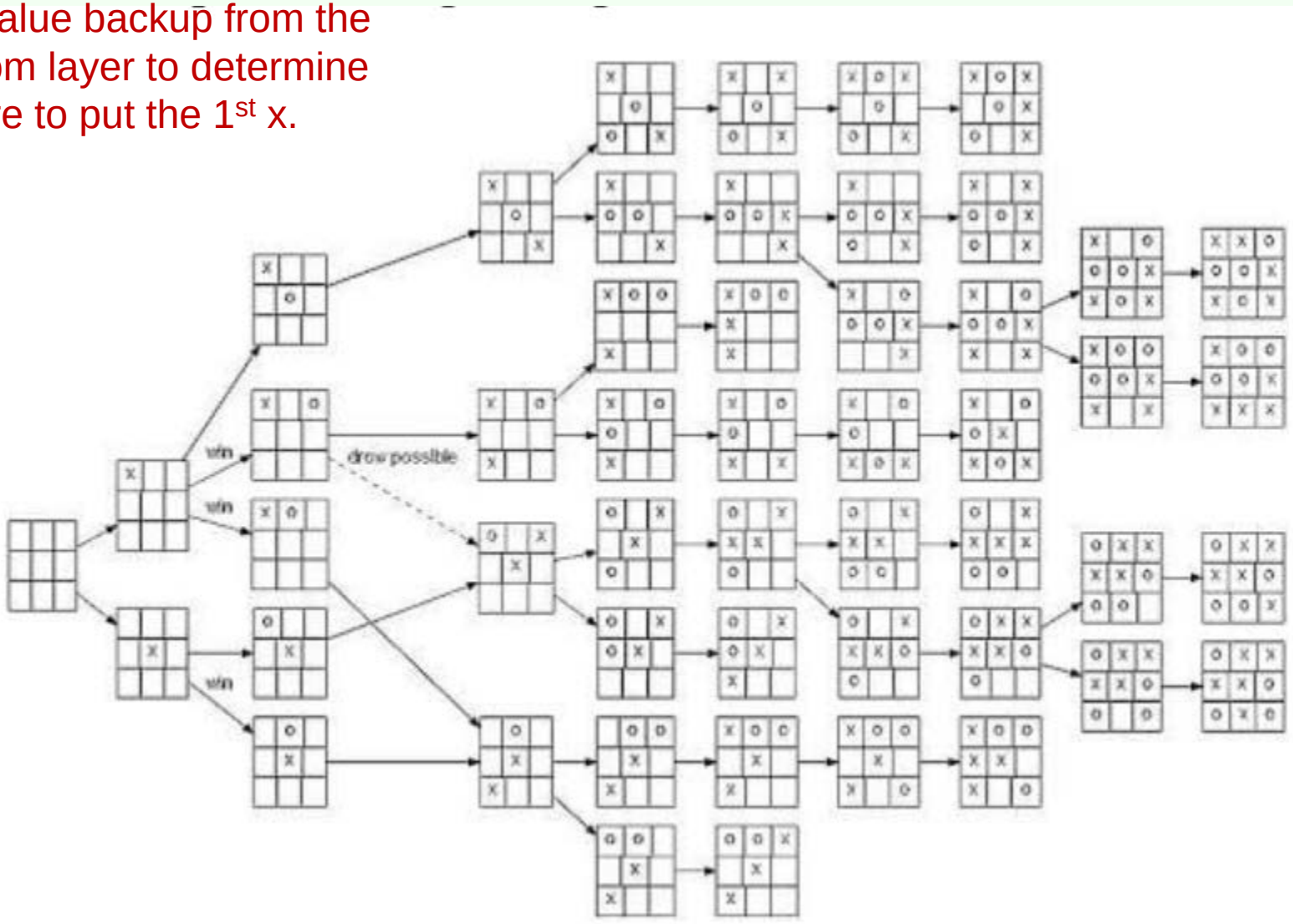
At level 8, do value backup. Since this level is “add o”, i.e., min’s move, he picks minimum scores from his children at level 8 to fill the scores at level 7.
(Question: at level 8, leafs have only scores -1. Why?)

At level 7, do value backup. Since this level is “add x”, i.e., max’s move, he picks maximum scores from his children to fill the scores at level 6.
(Question: at level 7, only scores +1. Why?)

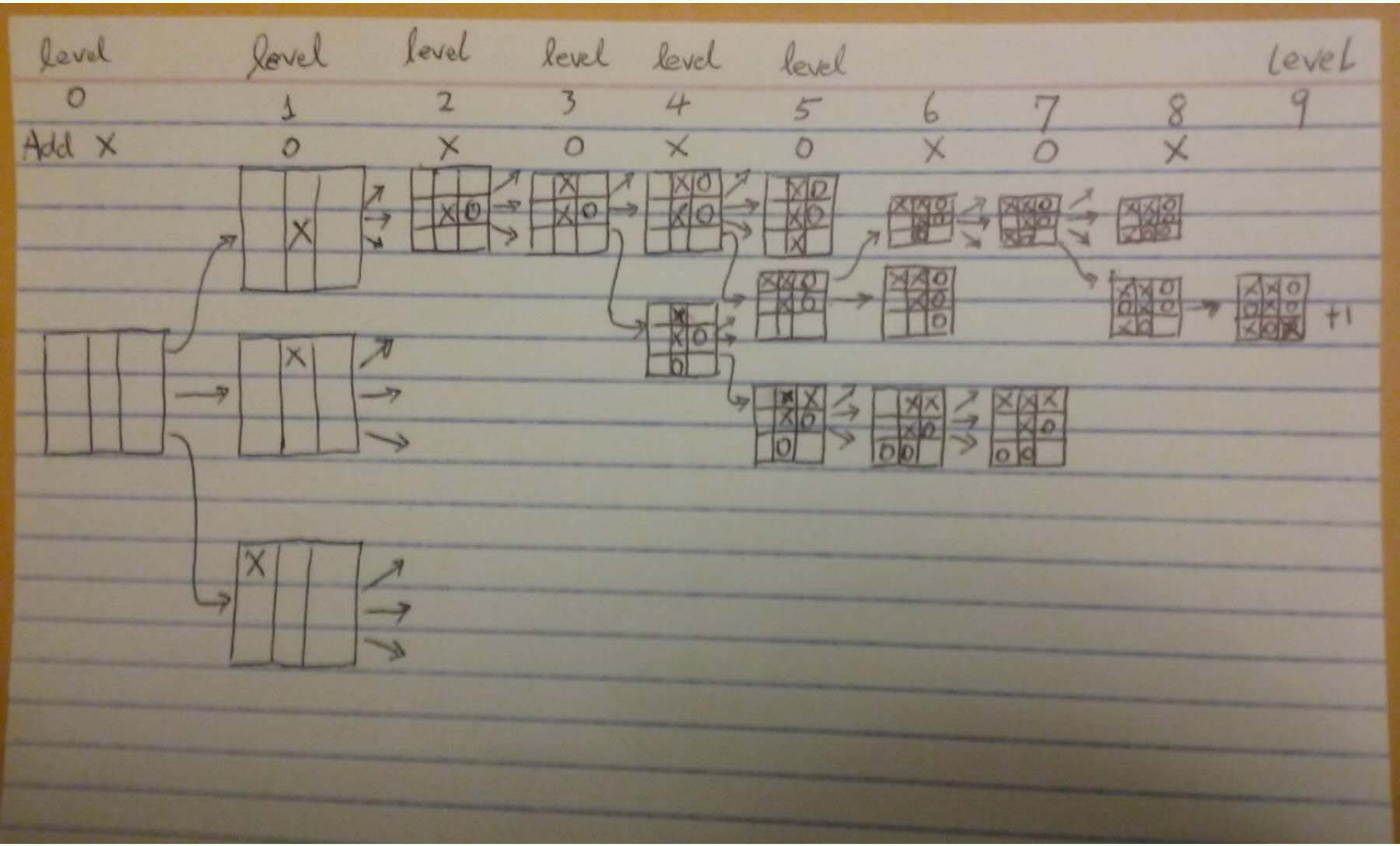
This is the minimax value backup. -→ minimax tree search


level 1
level 2
level 3
level 4
level 5
level 6
level 7
level 8
level 9
Tic Tac Toe
add o
add x
add o
add x
add o
add x
add o
add x
"x" win
"o" win
This branch terminate
"o" win terminate.
-1
computer search all branches
+1
terminate
+1
"x" win.
This branch terminated.
-1
terminate
tie
0

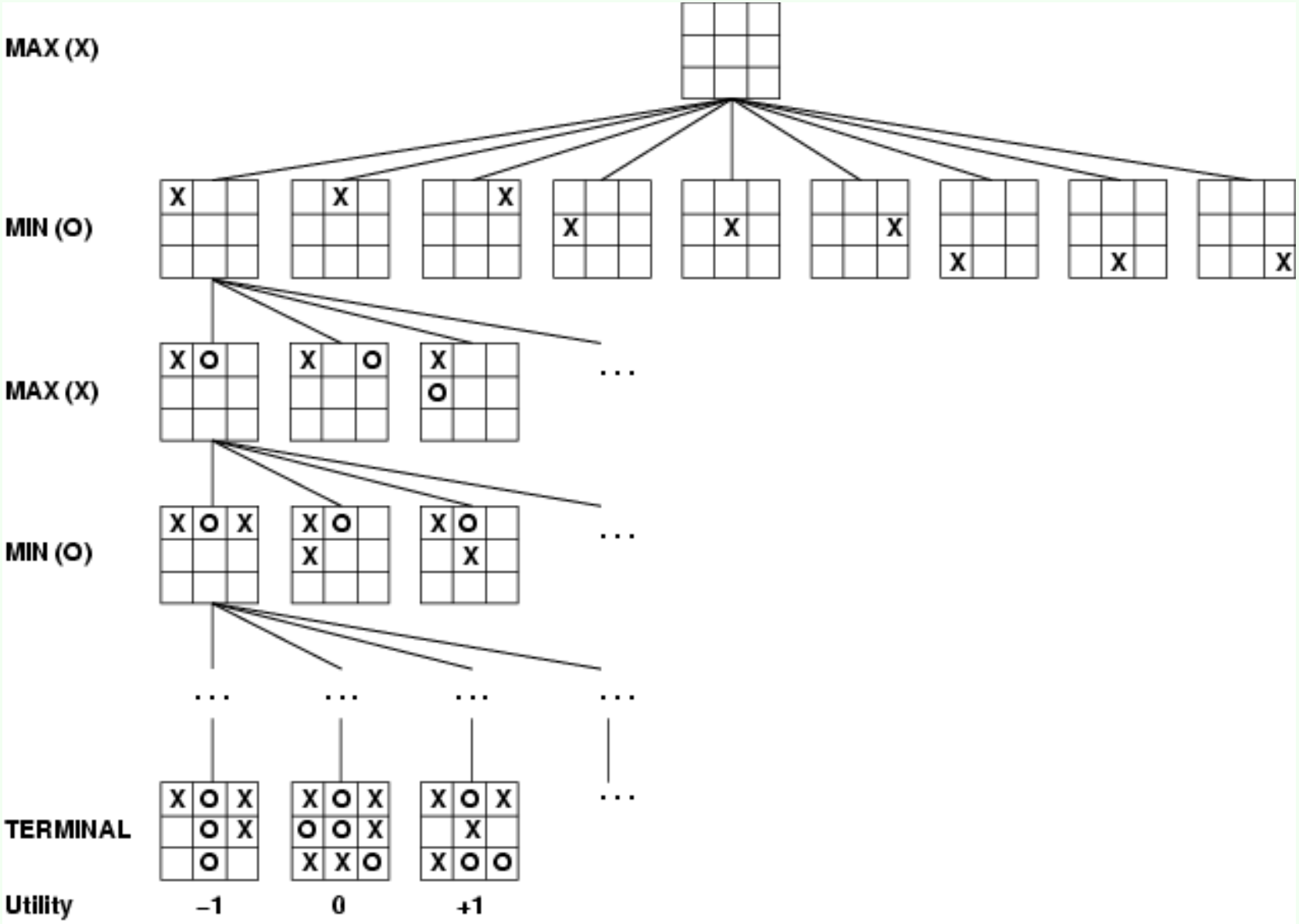
Homework: Game3:
Do value backup from the bottom layer to determine where to put the 1st x.



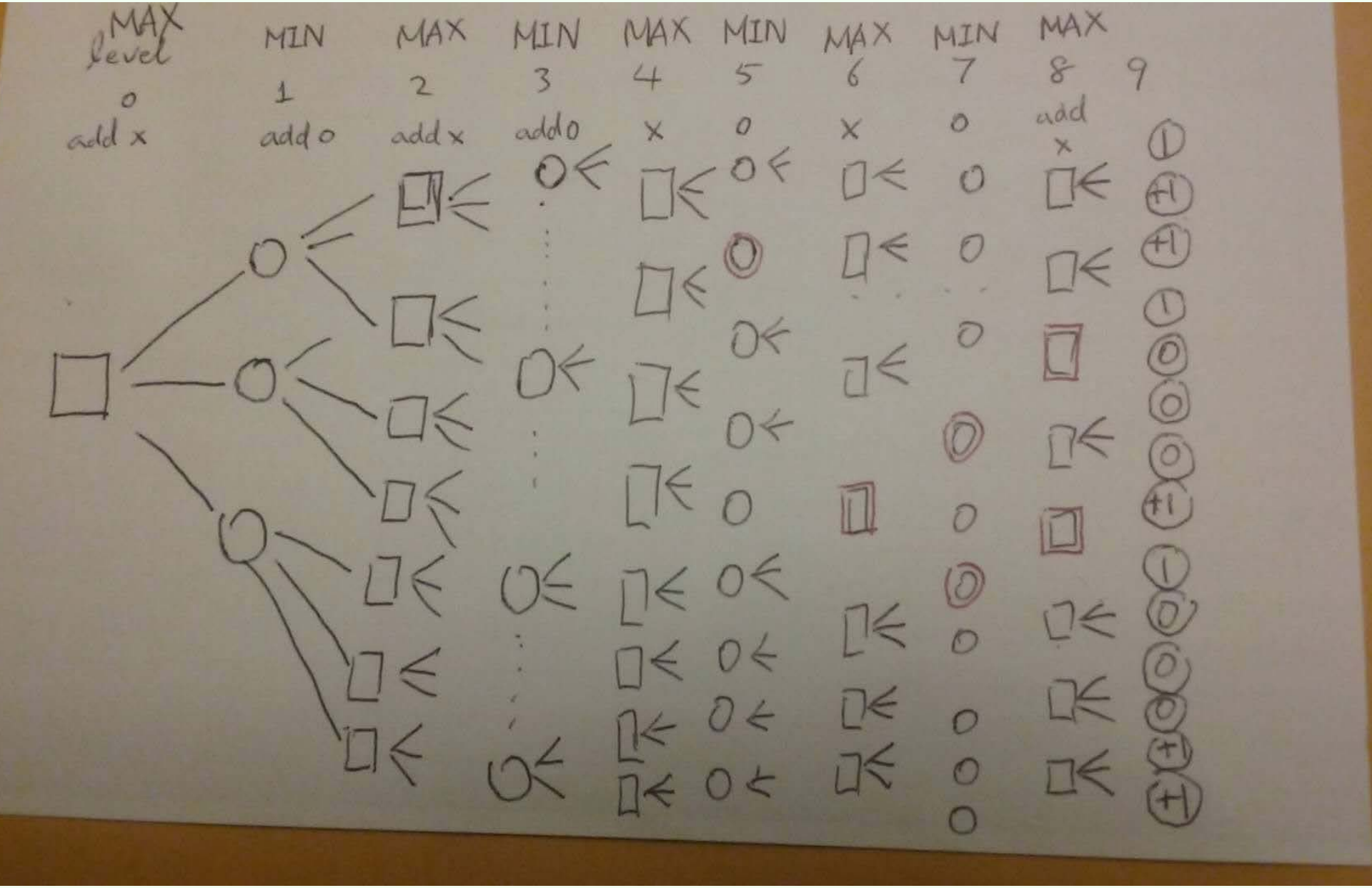
How to make the 1st move ?

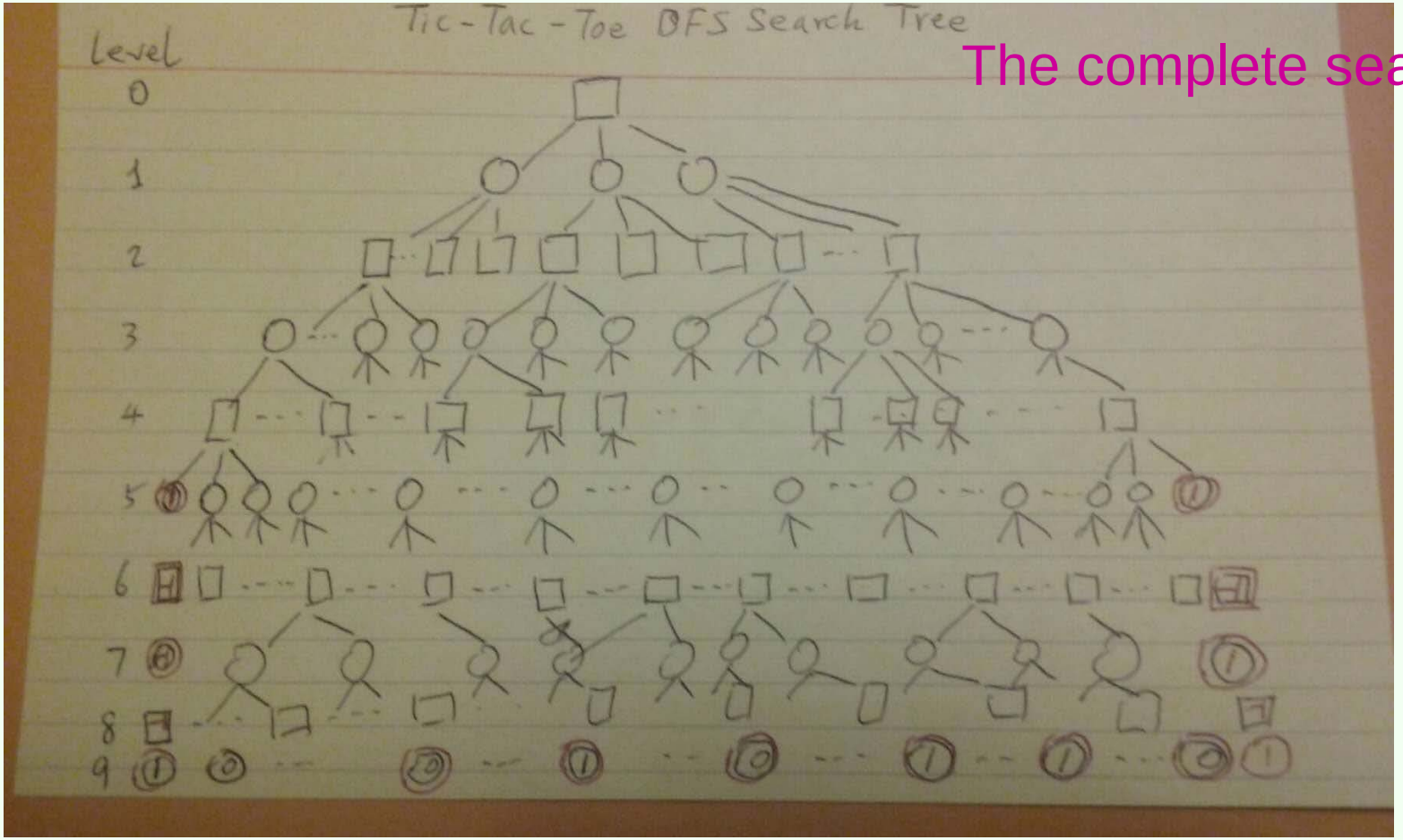

level 0
level 1
level 2
level 3
level 4
level 5
level 6
level 7
level 8
level 9
Add X
0
X
0
X
0
X
0
X
+1

The complete search tree



The complete search tree





The complete search tree

HOMEWORK:
Prove these properties

Tree property:

1. Leaf nodes occur at levels 5,6,7,8,9. Why?
2. Leaf nodes on level 5 has value +1. Why?
3. Leaf nodes on level 6 have value -1. Why?
4. Leaf nodes on level 9 (bottom) have values 0 or 1. Why?
5. How many leaf nodes on level 5?
6. How many leaf nodes on level 6?
7. How many leaf nodes on level 9?

How to solve the Tic-Tac-Toe using a computer?

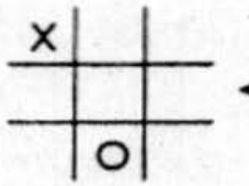
A better algorithm:

The complete search tree algorithm often requires too large space to store intermediate results.

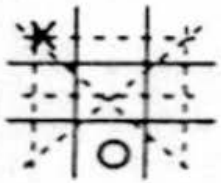
In practice, we use a depth-limited search. For example, we only search for 3 steps (moves). This limits the space requirement.

But if the game does not reach the end, we don’t know who will win. Thus we need to define a heuristic score to evaluate the state/configuration.

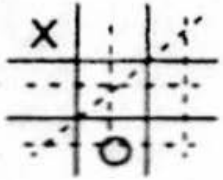
The static evaluation function heuristic



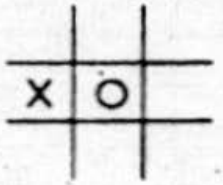
X has 6 possible win paths:



O has 5 possible wins:

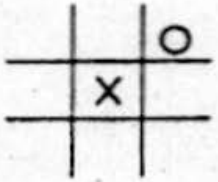


E(n) = 6 – 5 = 1



X has 4 possible win paths;
O has 6 possible wins

E(n) = 4 – 6 = –2



X has 5 possible win paths;
O has 4 possible wins

E(n) = 5 – 4 = 1

Heuristic is E(n) = M(n) – O(n)

where M(n) is the total of My possible winning lines

O(n) is total of Opponent's possible winning lines

Heuristic Score

$e(p)$ = (number of complete rows, columns, or diagonals that are still open for *MAX*) – (number of complete rows, columns, or diagonals that are still open for *MIN*)

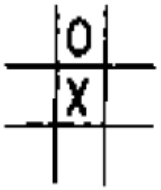
If p is a win for *MAX*,

$e(p) = \infty$ (I use ∞ here to denote a very large positive number)

If p is a win for *MIN*,

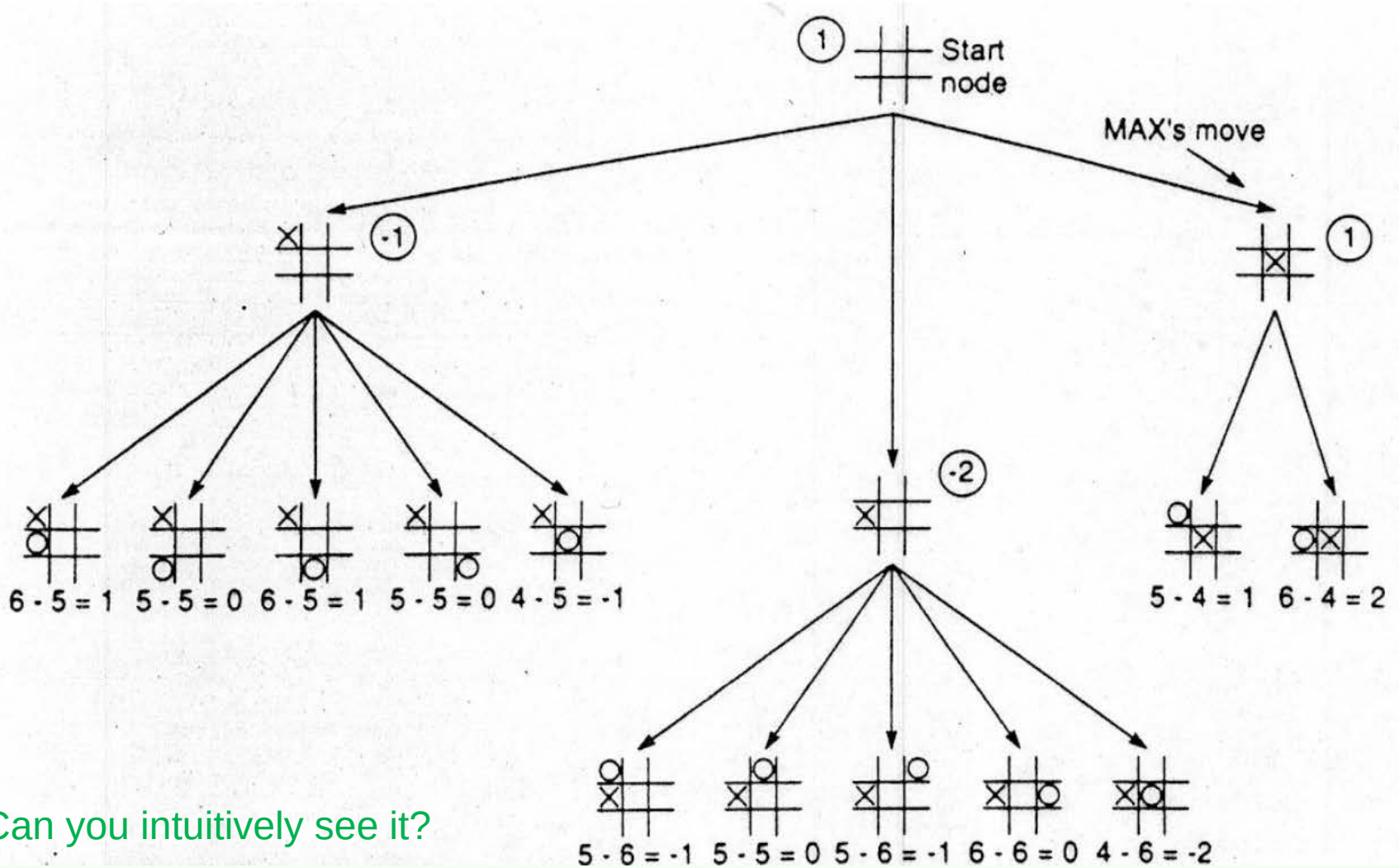
$e(p) = -\infty$

Thus, if p is



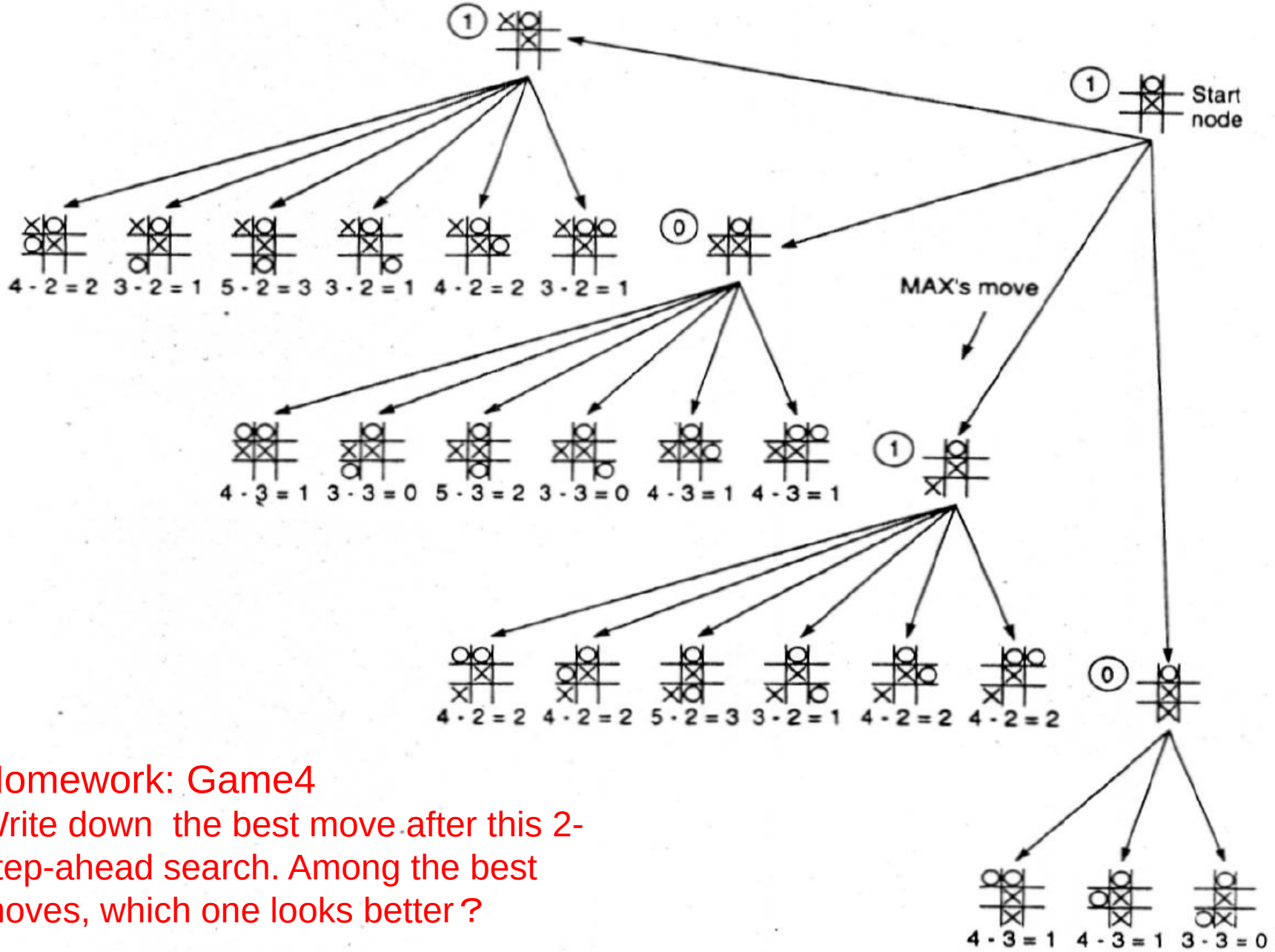
we have $e(p) = 6 - 4 = 2$.

2-step ahead search for the 1st move



Can you intuitively see it?

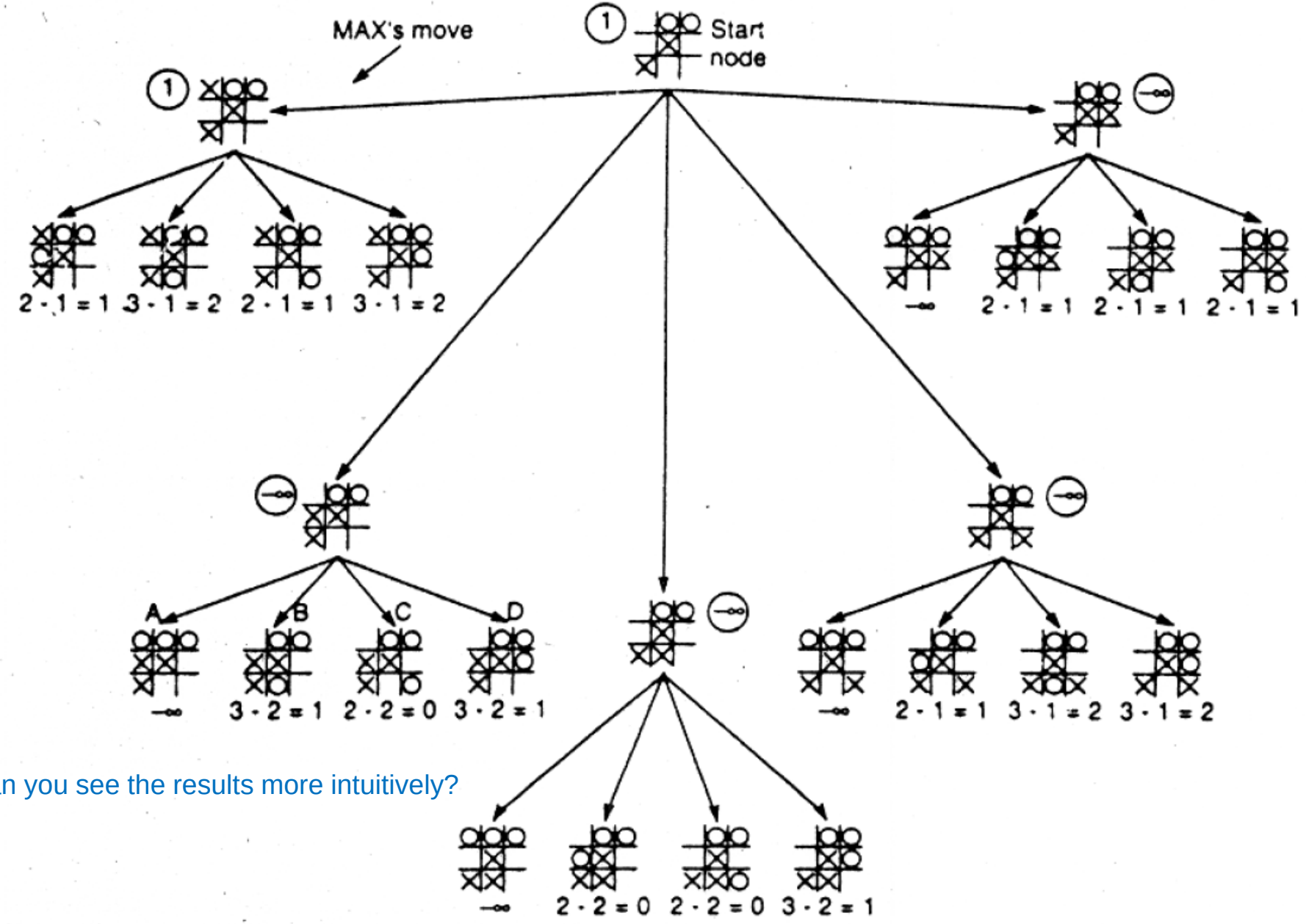
2-step-ahead search using heuristic evaluation function



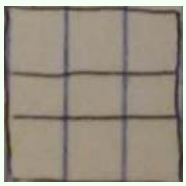
Homework: Game4

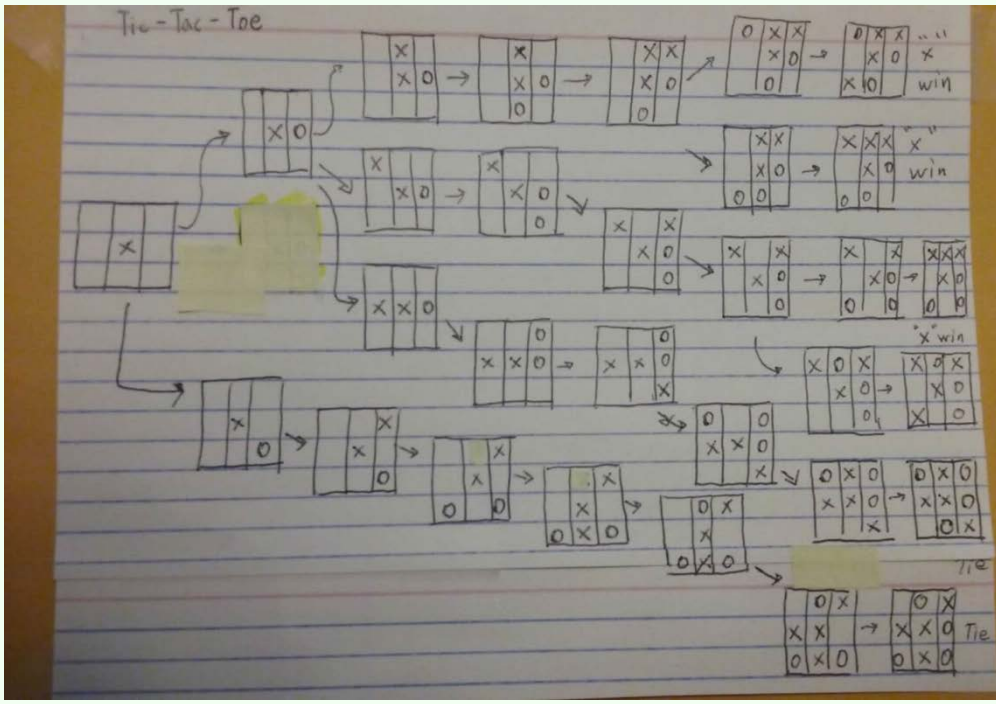
Write down the best move after this 2-step-ahead search. Among the best moves, which one looks better?

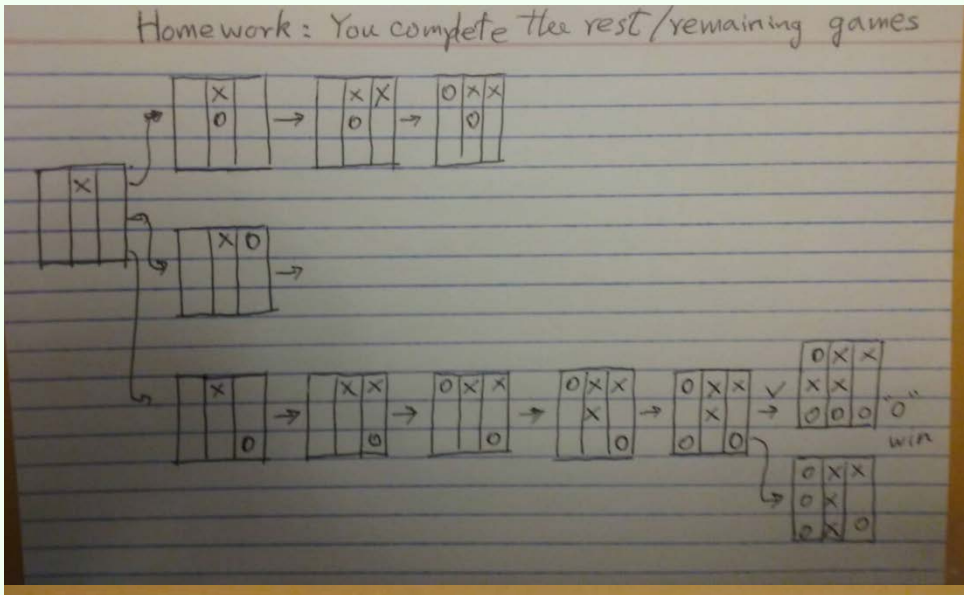
2-step-ahead search using heuristic evaluation function

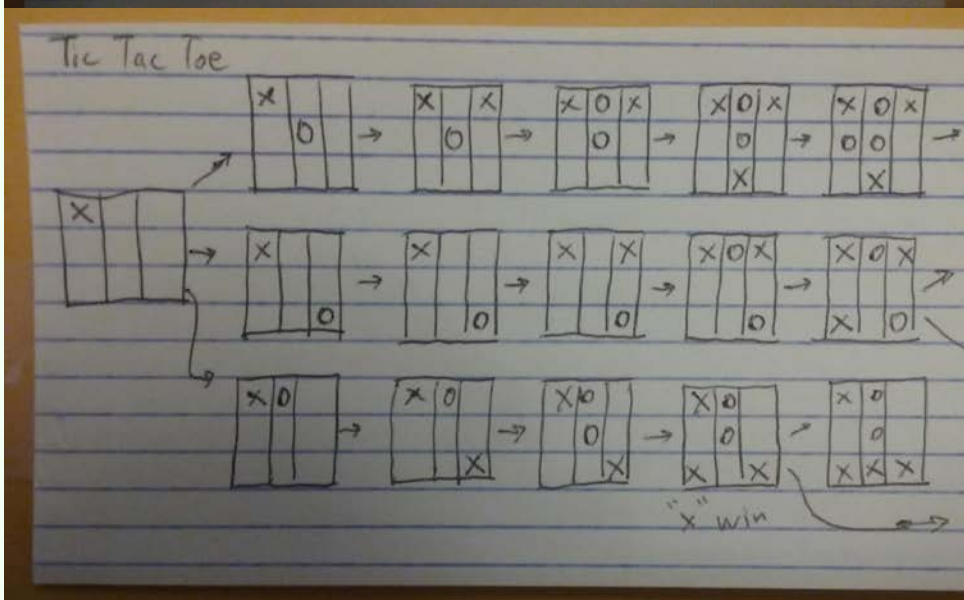


Can you see the results more intuitively?



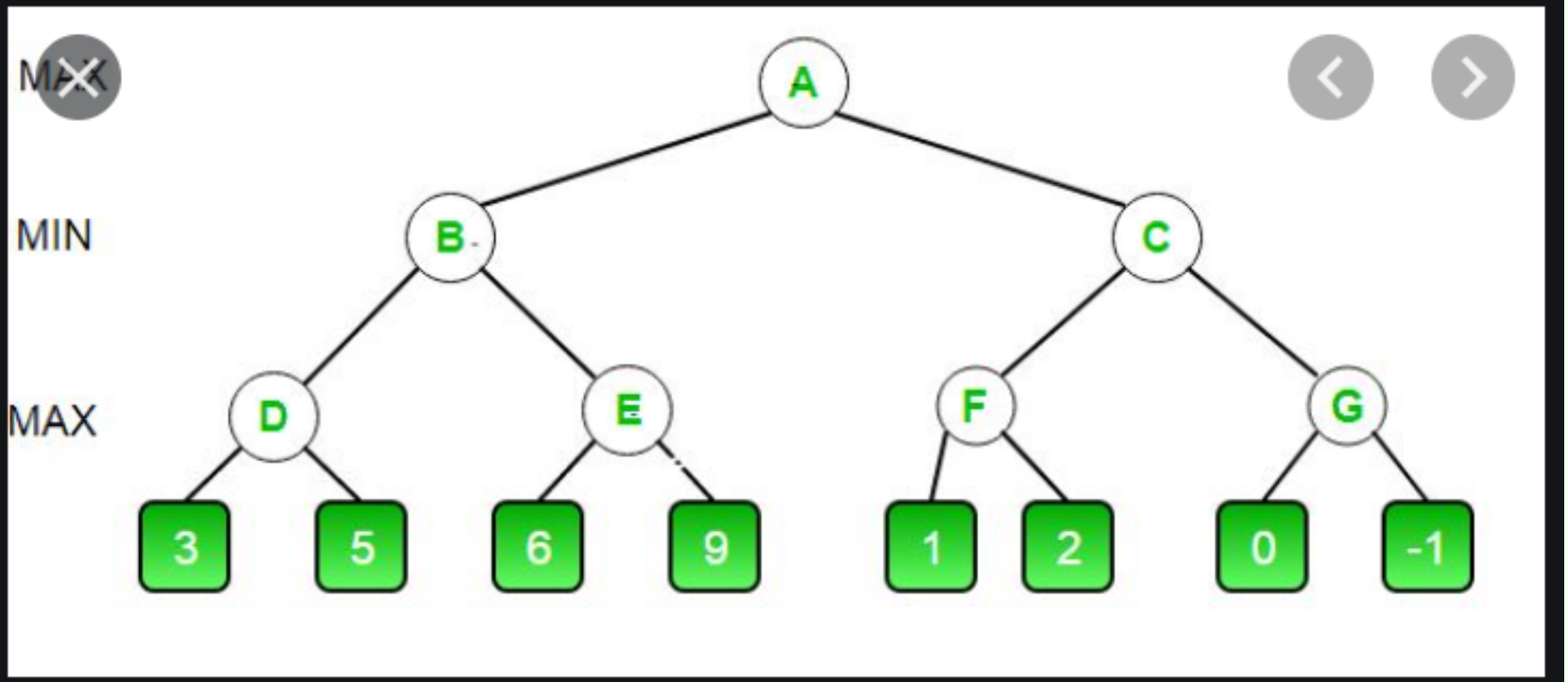

Tic-Tac-Toe
"O" win
"X" win
"X" win
Tie
Tie


Homework: You complete the rest/remaining games
"O" win


Tic Tac Toe
"X" win

The Minimax Tree Search Algorithm for Game Play

Search best move in the Tic-Tac-Toe DFS Search Tree
Becomes a minimax search on the tree

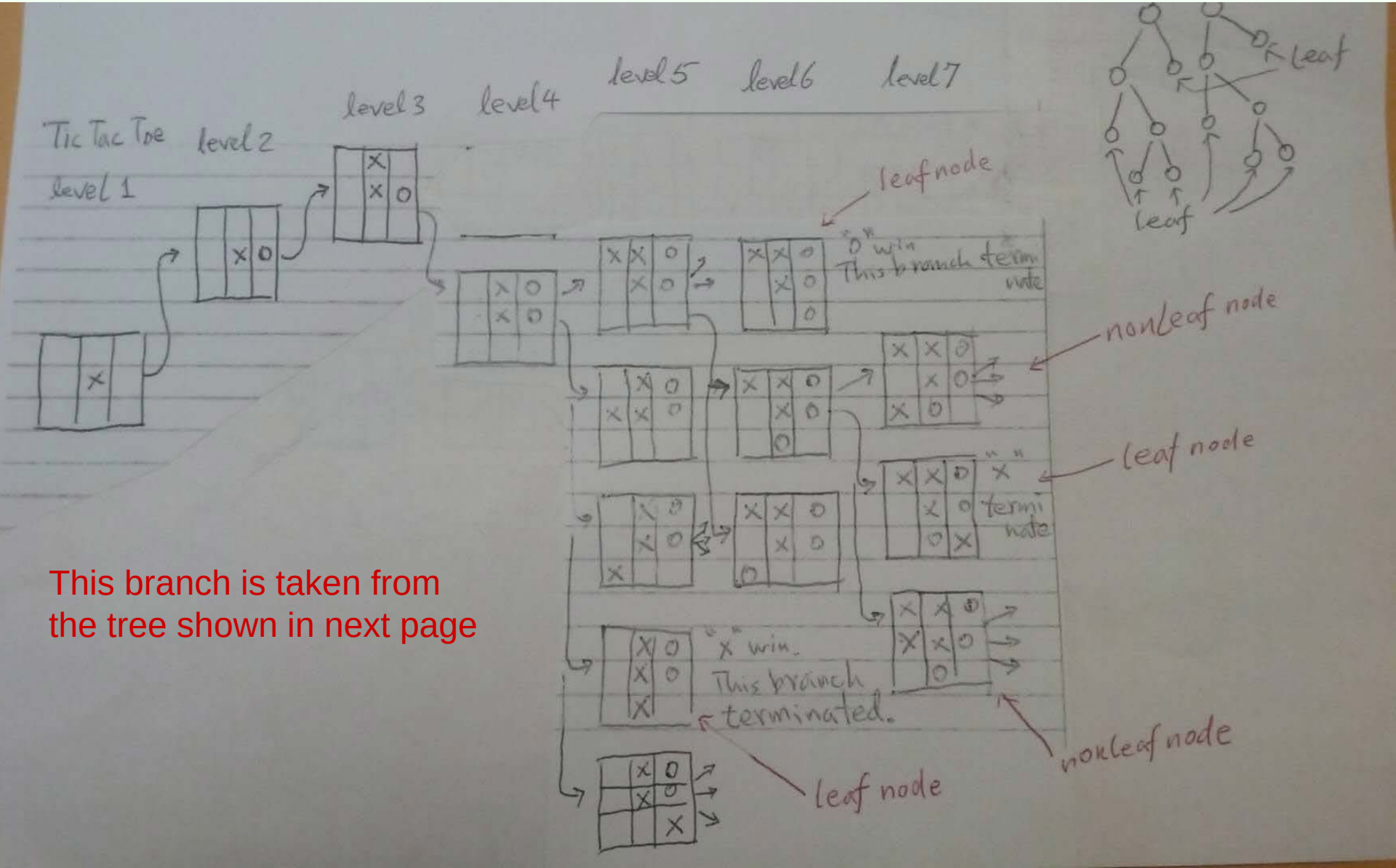


GeeksforGeeks

Minimax Algorithm in Game Theory | Set 4 (Alp...

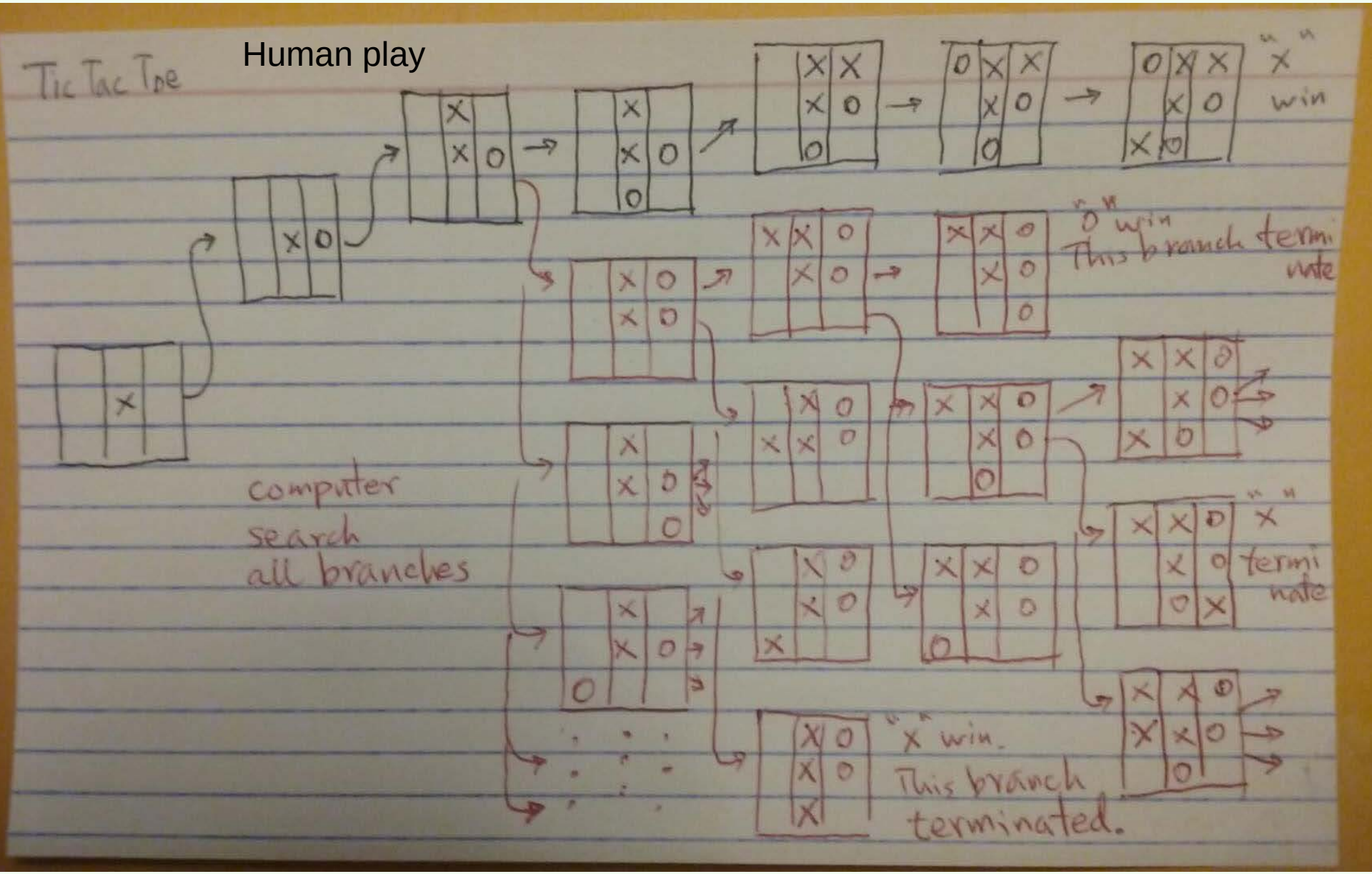
Homework. A branch from the Tic-Tac-Toe DFS Search Tree (TTTDST).

(a) Draw all states/nodes at level 6. (b) At level 7, from each level-6 state *s* , indicates how many children *s* has, including LN(leaf node) and NLN(Nonleaf node) (c) From these results, for the entire TTTST, estimate how many leaf nodes at level 5 and leaf nodes at level 6.

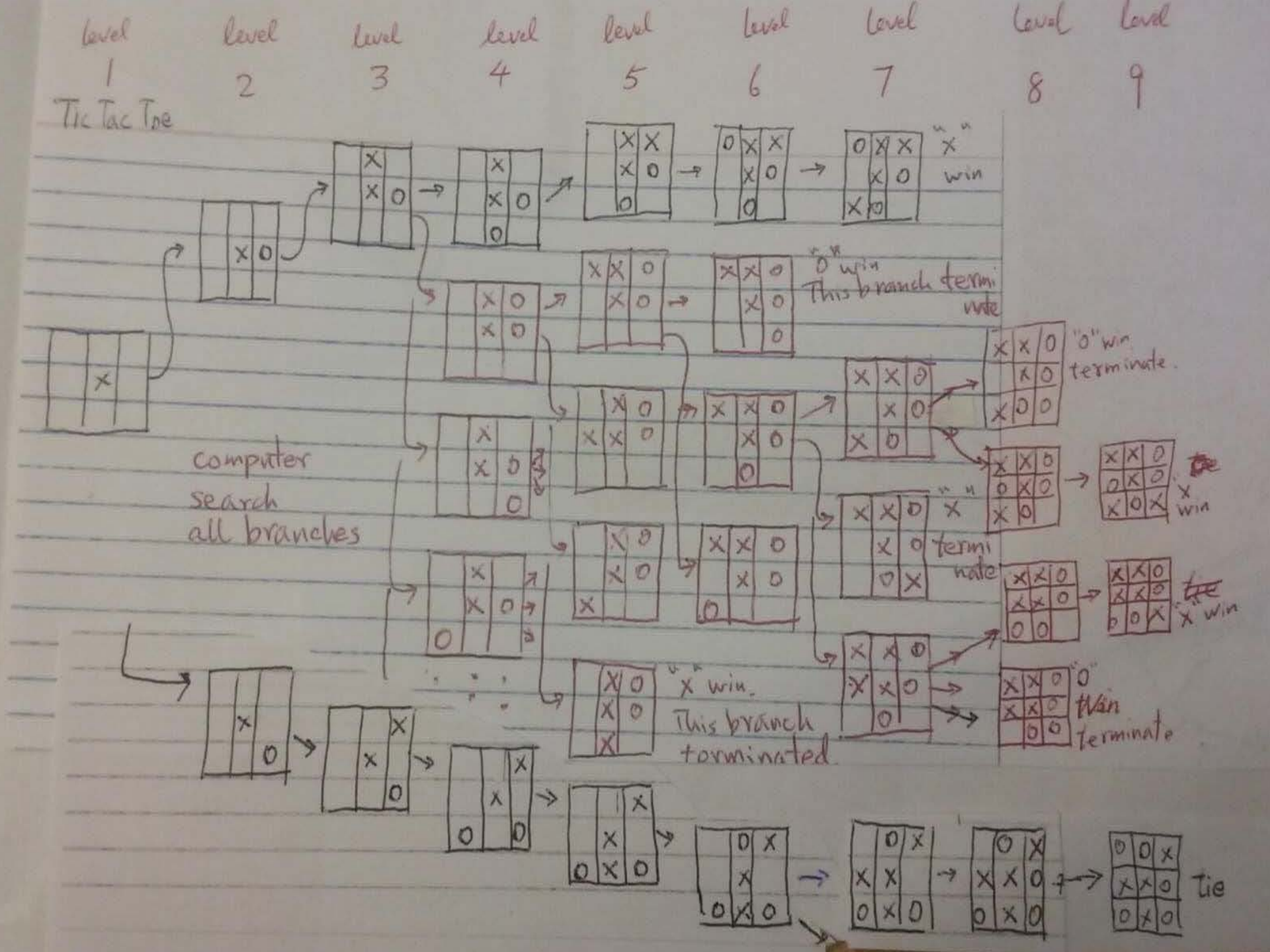


This branch is taken from the tree shown in next page

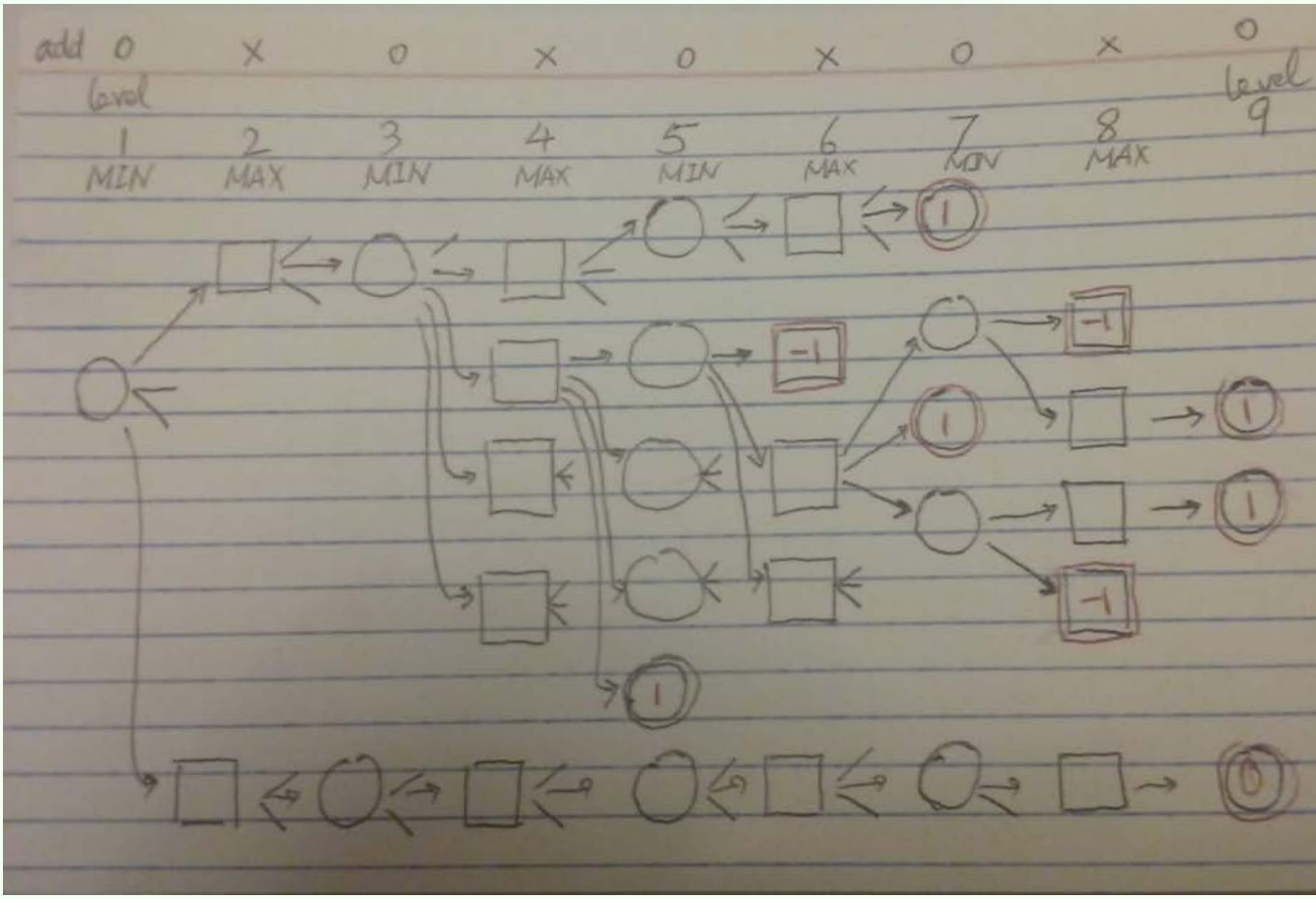
Tic-Tac-Toe DFS Search Tree

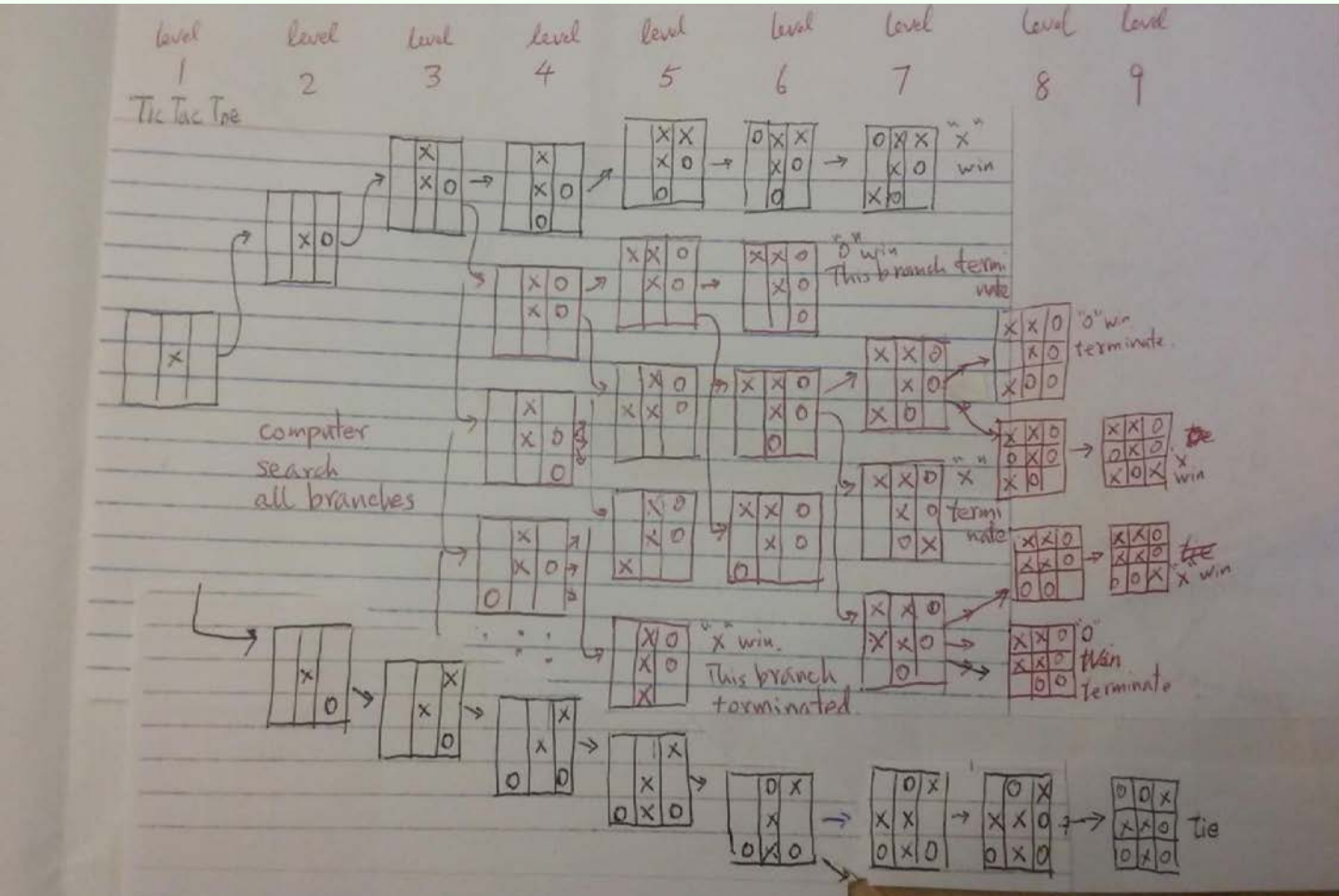


Search best move in the Tic-Tac-Toe BFS Search Tree
Becomes a minimax search on the tree

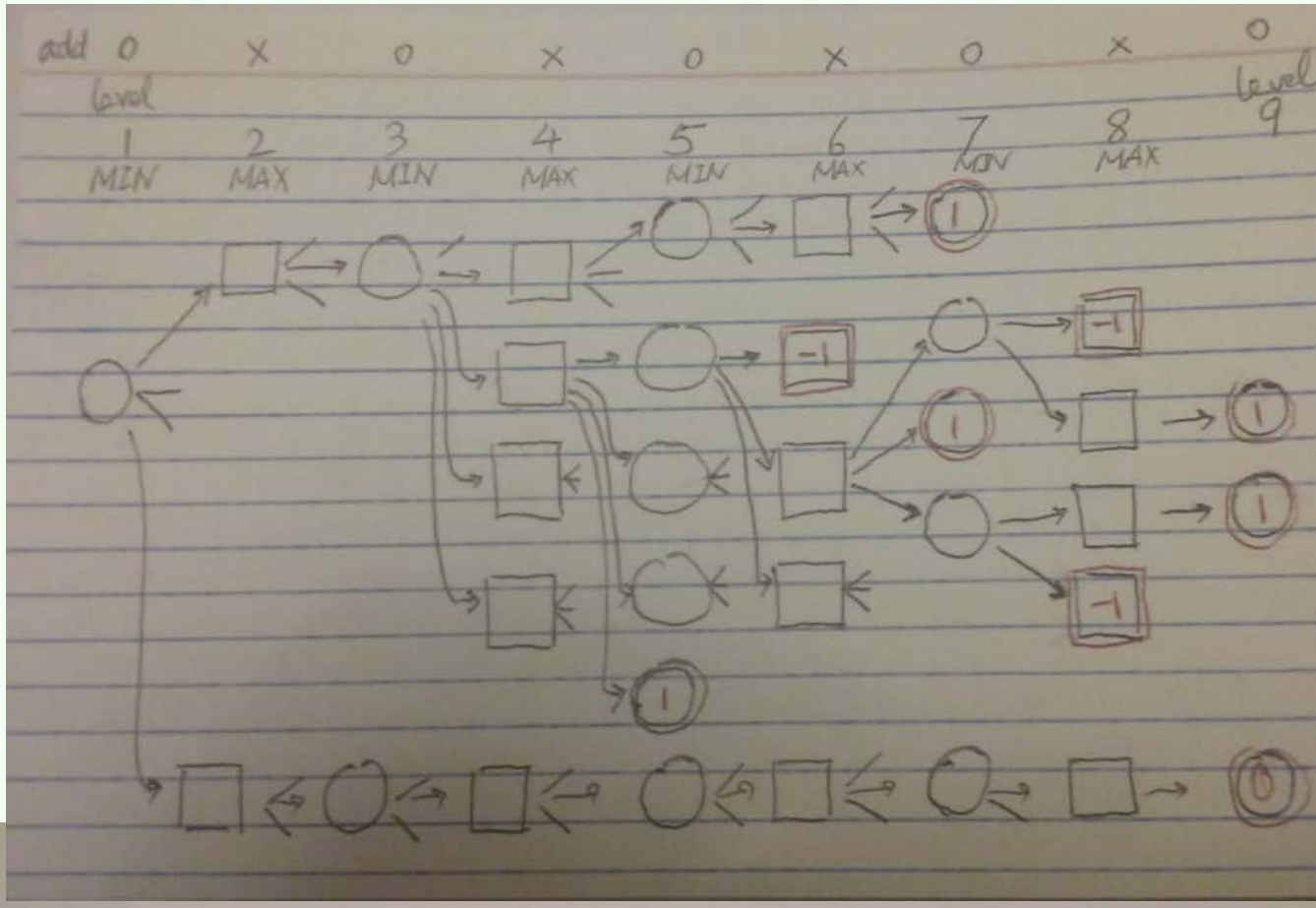

level 1
level 2
level 3
level 4
level 5
level 6
level 7
level 8
level 9
Tic Tac Toe
"X" win
"O" win
This branch terminate
"O" win terminate.
computer search all branches
terminate
X win
tie
"X" win.
This branch terminated.
"O" win terminate
tie

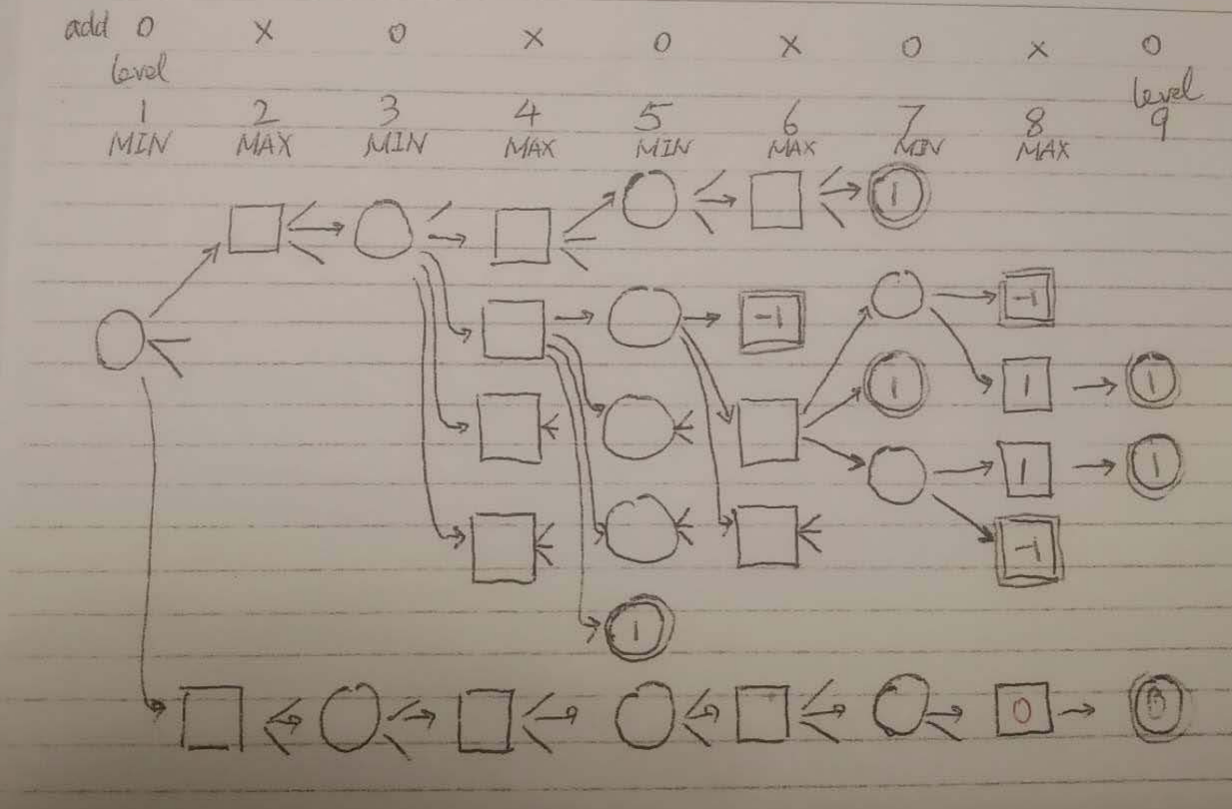
Why it becomes a minimax tree ?



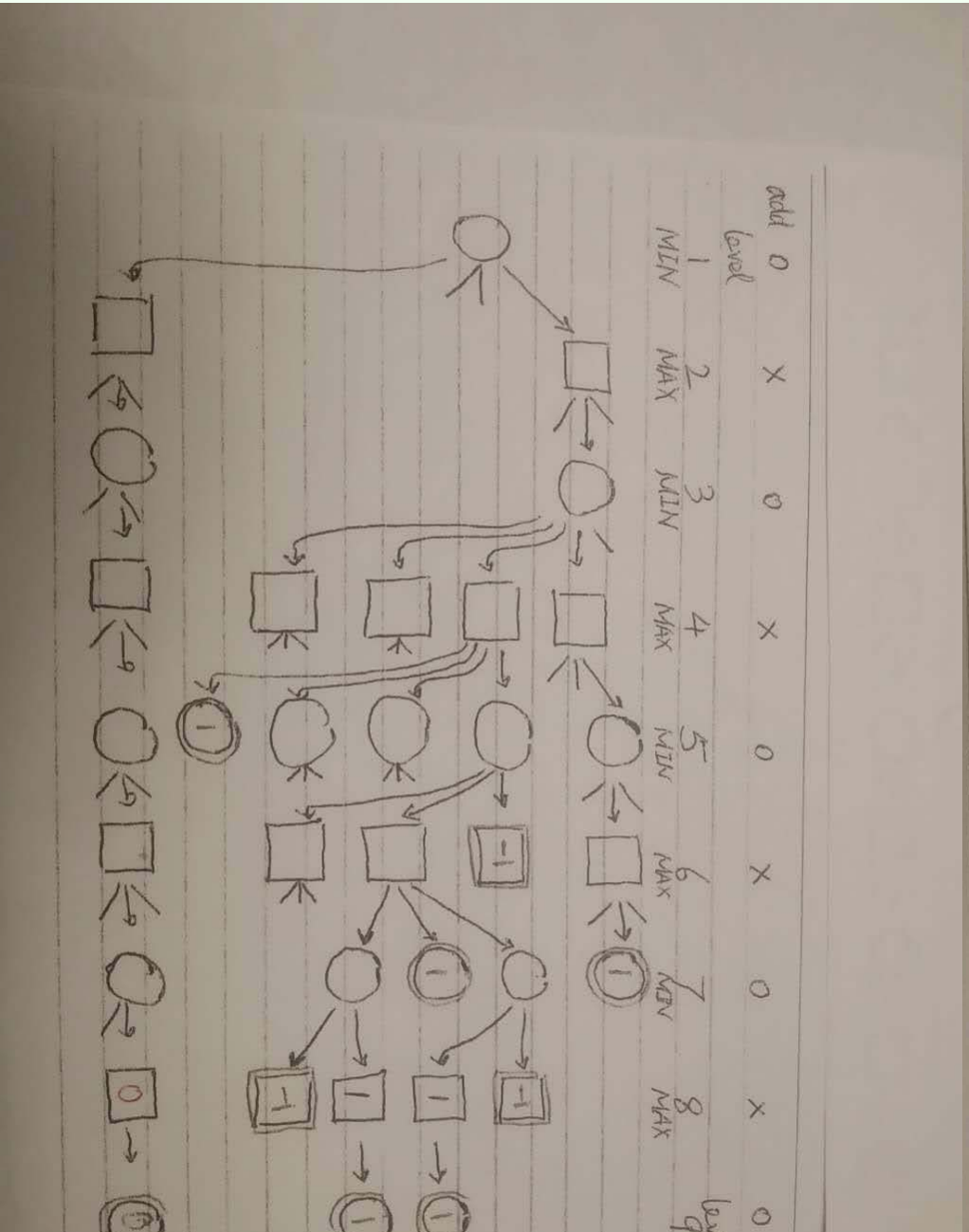


How to evaluate a minimax tree ?

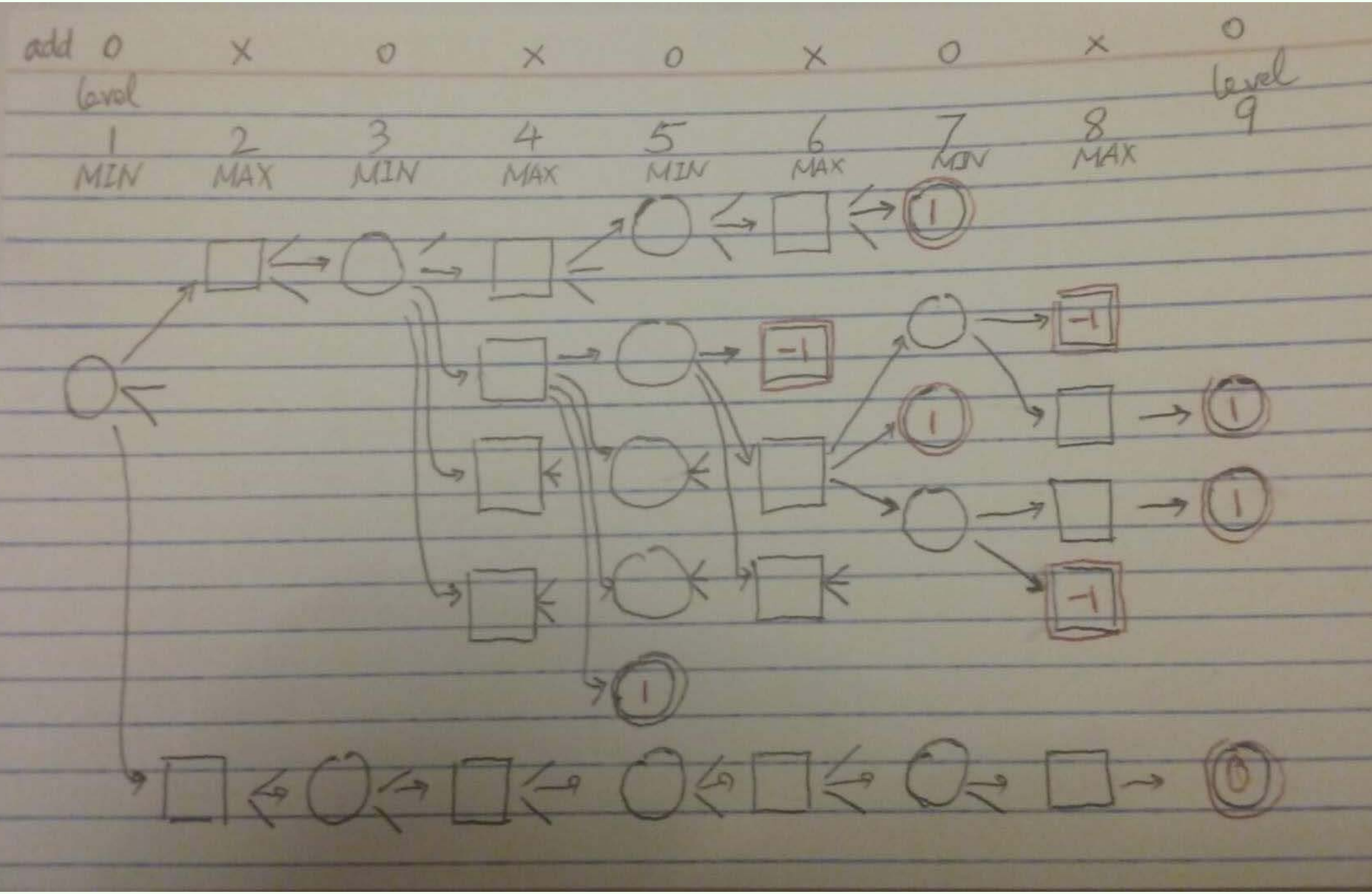




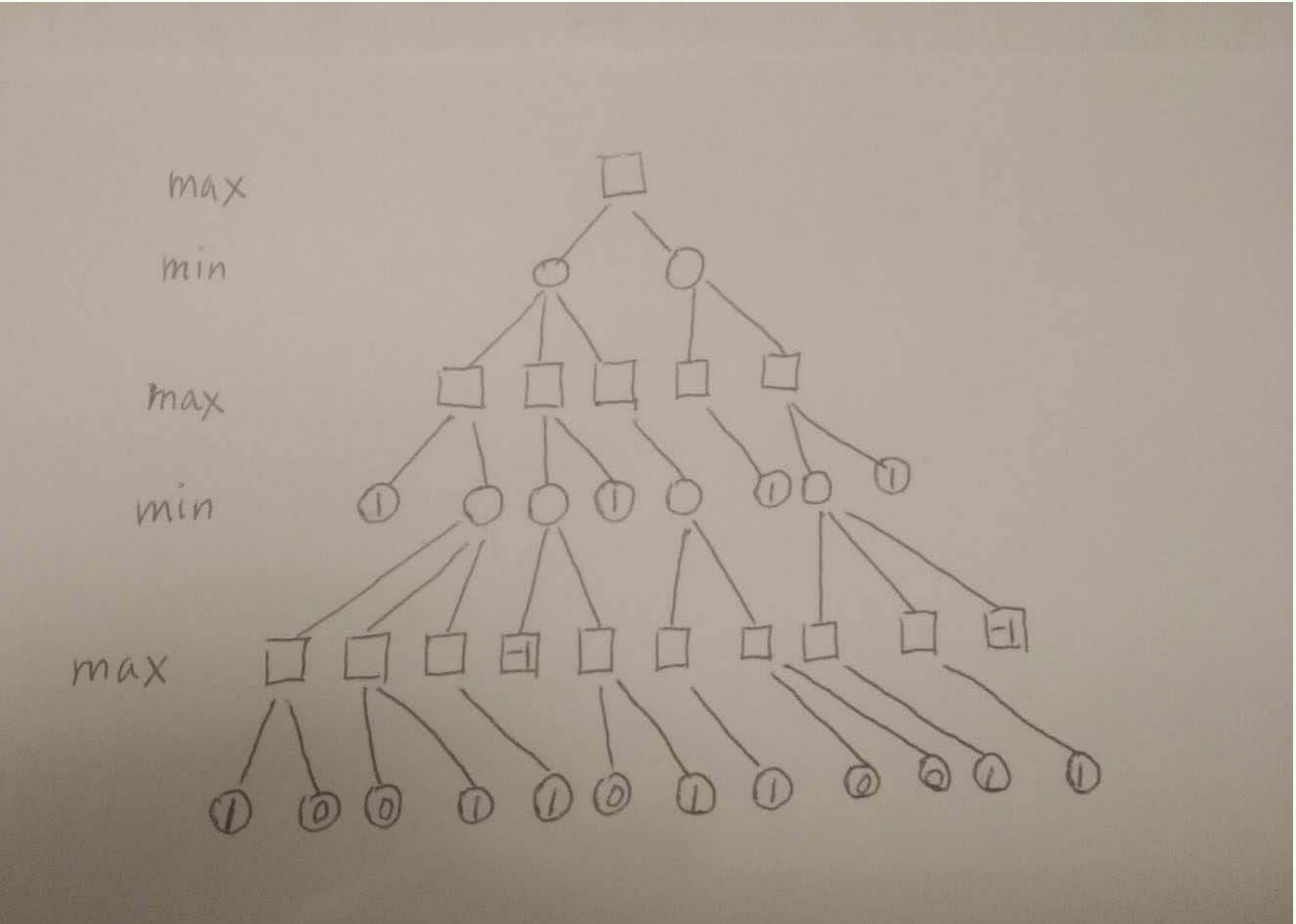
How to evaluate a minimax tree ?



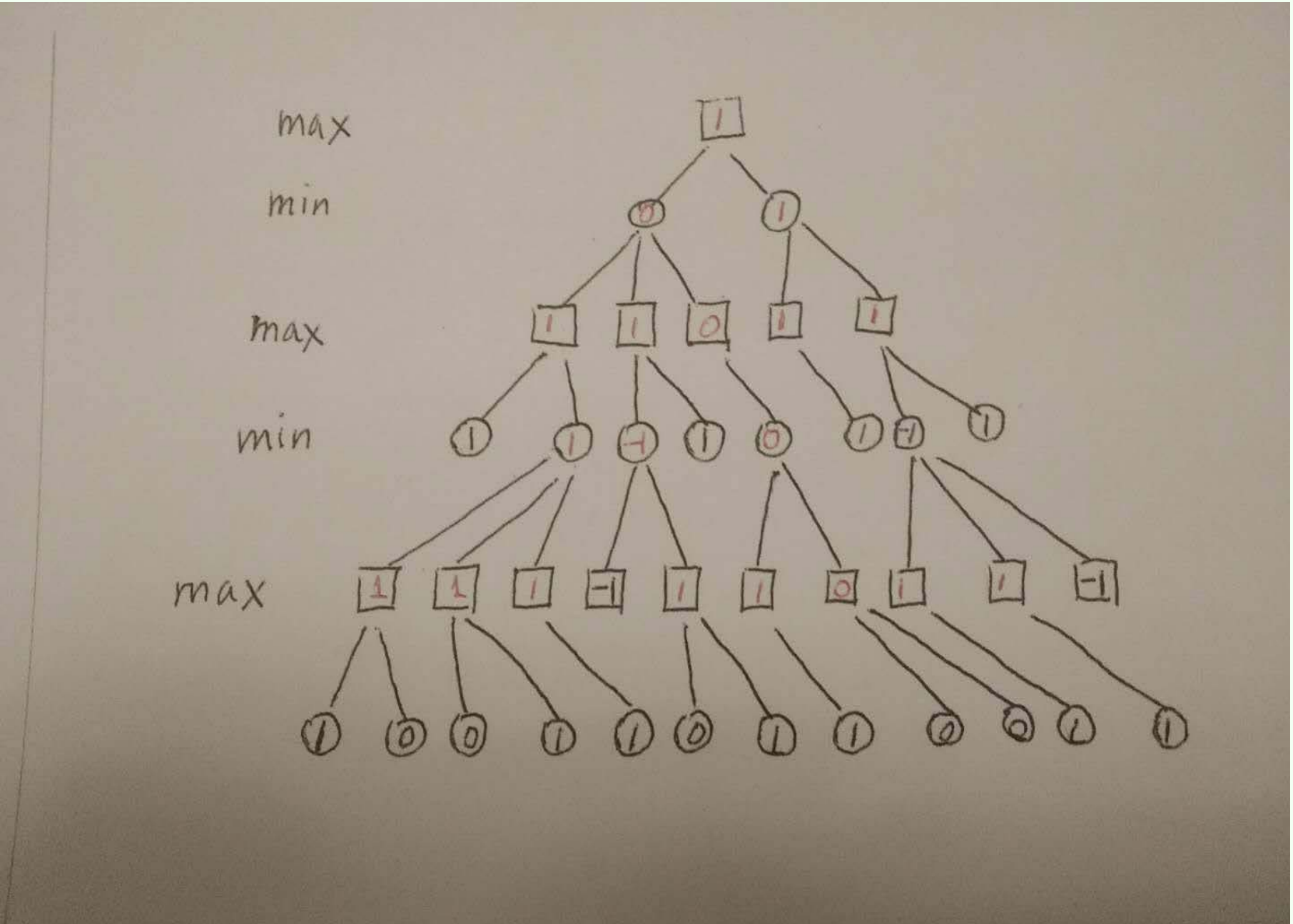
Minimax Search Tree



Evaluation of Minimax Search Tree

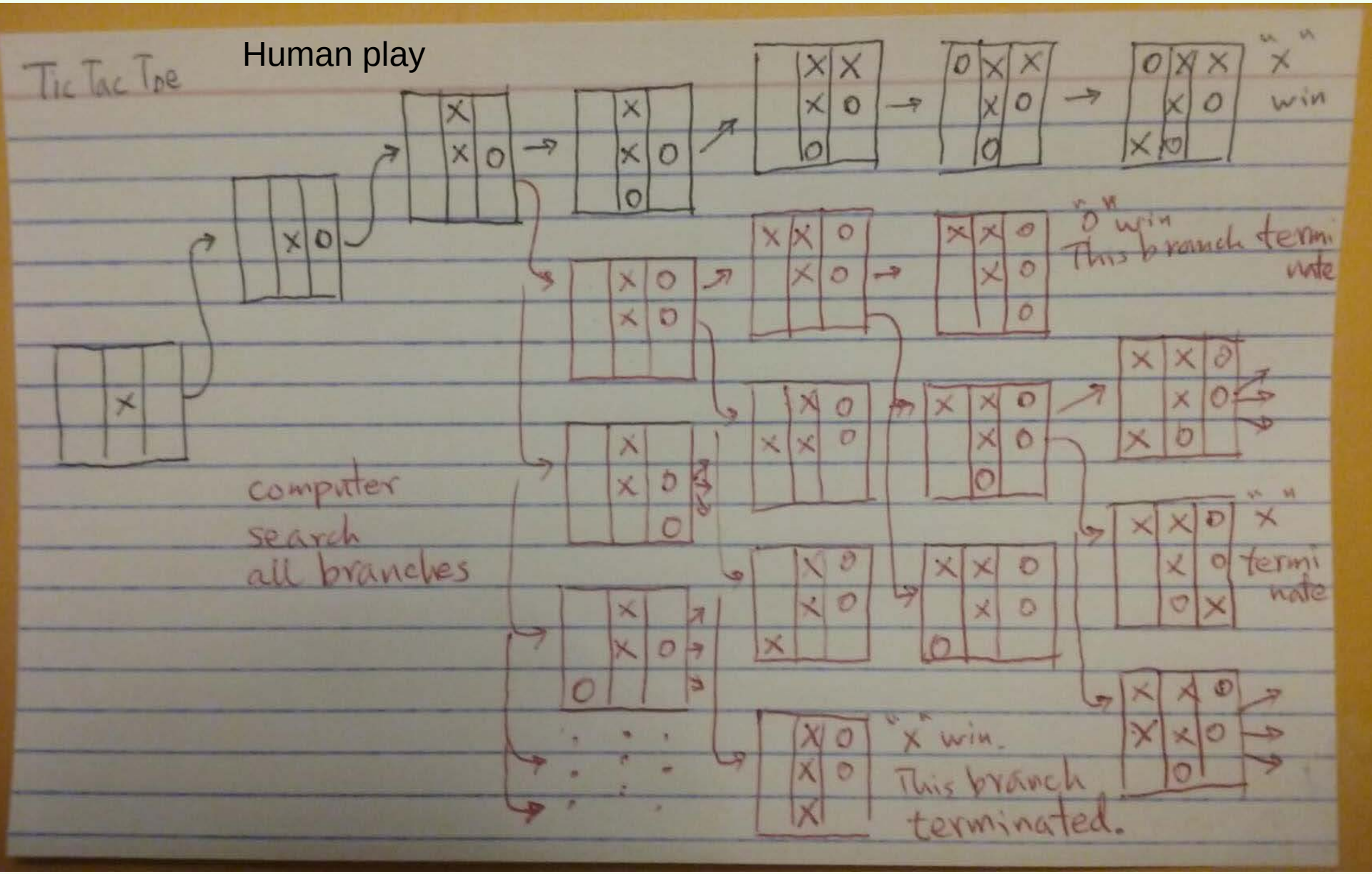


Bottom-up Evaluation of Minimax Search Tree



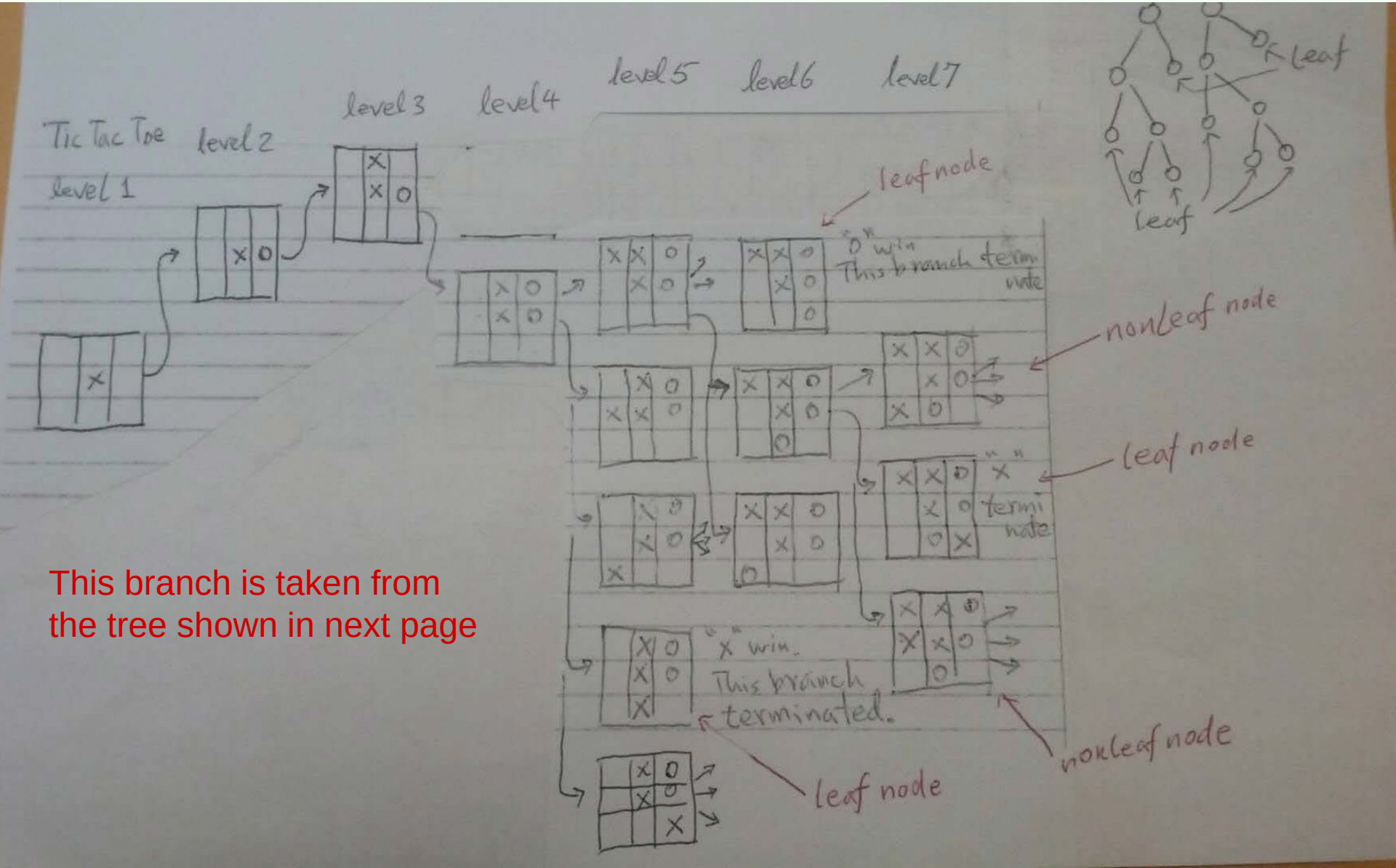
Homework: to get a sense of how large is the search tree

Tic-Tac-Toe DFS Search Tree



Homework. A branch from the Tic-Tac-Toe DFS Search Tree (TTTDST).

(a) Draw all states/nodes at level 6. (b) At level 7, from each level-6 state *s* , indicates how many children *s* has, including LN(leaf node) and NLN(Nonleaf node) (c) From these results, for the entire TTTST, estimate how many leaf nodes at level 5 and leaf nodes at level 6.

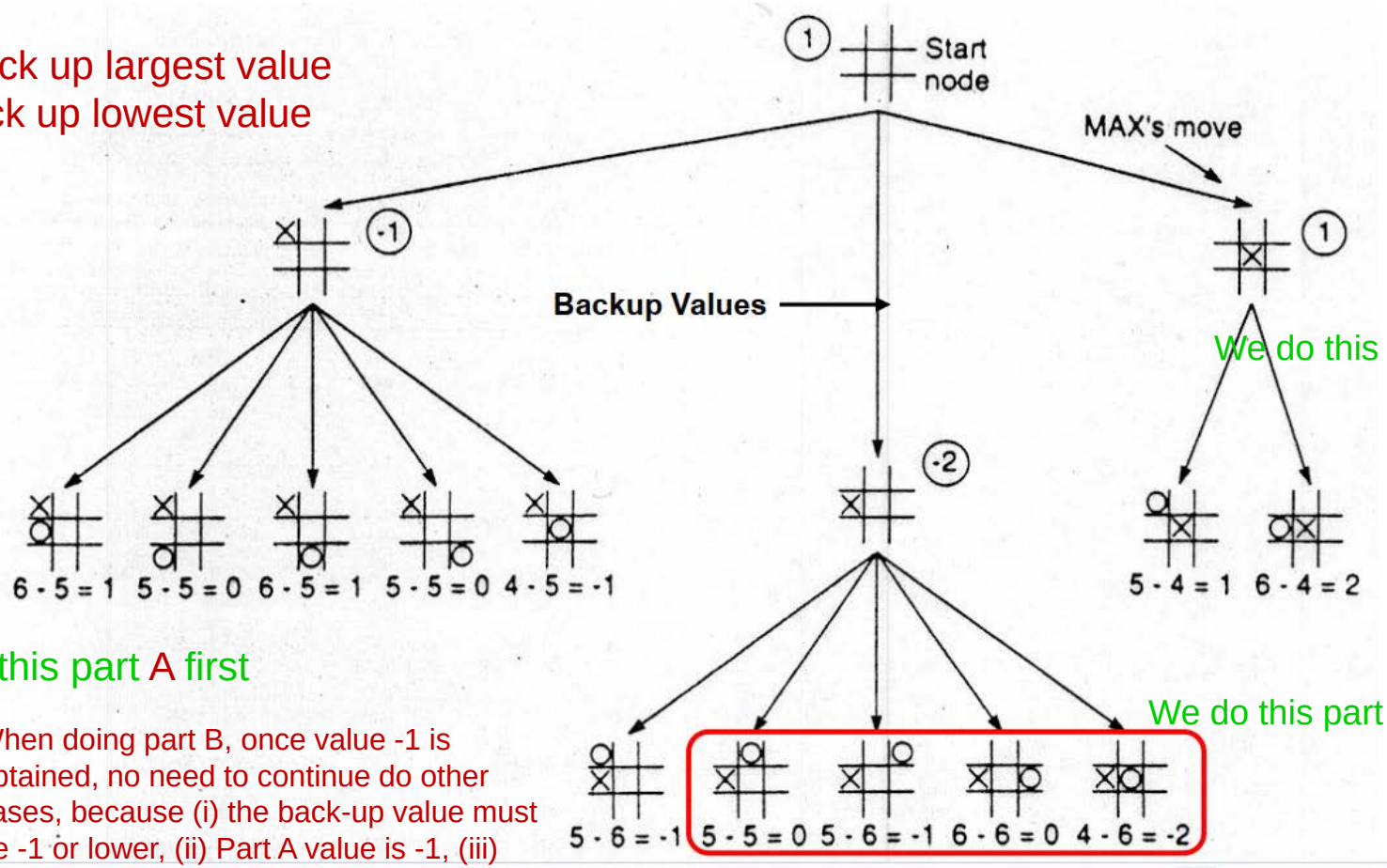


This branch is taken from the tree shown in next page

To speedup the evaluation of the search tree, we use alpha-beta pruning

Tic-Tac-Toe Example with Alpha-Beta Pruning

MAX back up largest value
MIN back up lowest value

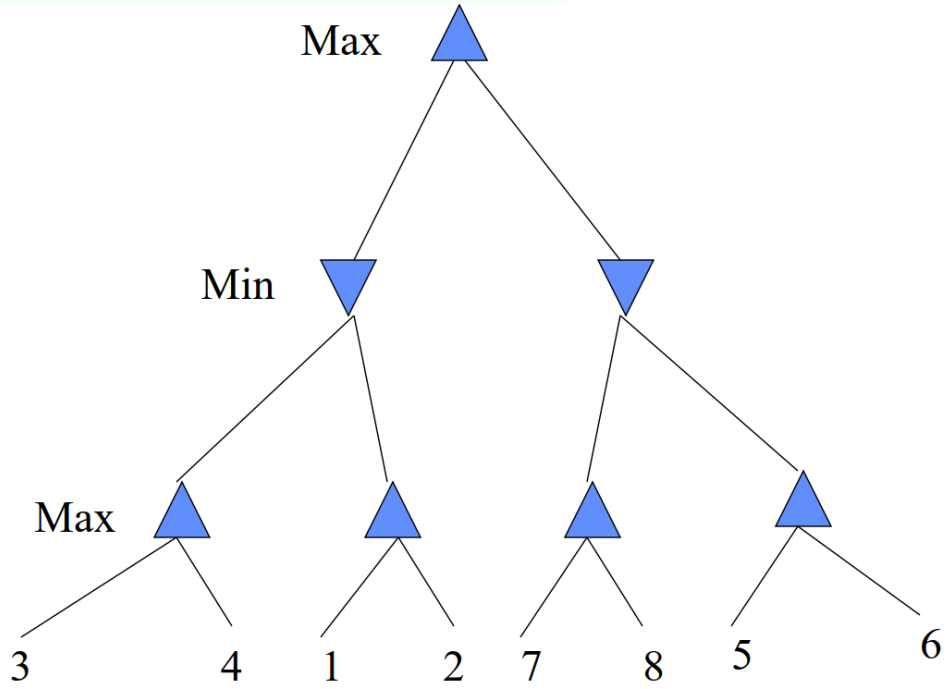


We do this part A first

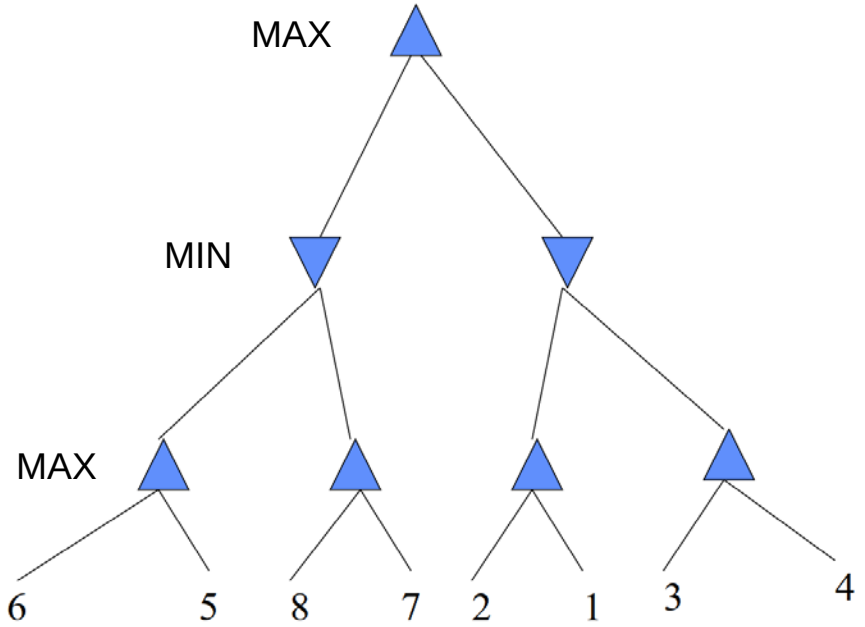
We do this part B second

We do this part C third

When doing part B, once value -1 is obtained, no need to continue do other cases, because (i) the back-up value must be -1 or lower, (ii) Part A value is -1, (iii) MAX back-up largest value



Evaluate Value-Back-Up using minimax algorithm, with alpha-beta pruning on these two cases.



Game Tree Size

- **Tic-Tac-Toe**
  - b ≈ 5 legal actions per state on average, total of 9 plies in game.
    - "ply" = one action by one player, "move" = two plies.
  - 5^9 = 1,953,125
  - 9! = 362,880 (Computer goes first)
  - 8! = 40,320 (Computer goes second)
  - → **exact solution quite reasonable**

- **Chess**
  - b ≈ 35 (approximate average branching factor)
  - d ≈ 100 (depth of game tree for "typical" game)
  - $b^d \approx 35^{100} \approx 10^{154}$ nodes!!
  - → **exact solution completely infeasible**

Look-ahead Chess Play Strategy

The tic-tac-toe game tree algorithm we had analyzed is based on searching all steps towards the end

In most games such chess, chinese chess, go, etc, the search tree is way too huge for searching towards the end. The common way to proceed is to search the next 3-4 steps (3 steps look-ahead).

A world champion can see 5 steps ahead. A normal good player can see 2-3 steps ahead.

We examine the 1-step look-ahead algorithm on Tic-tac-toe

Checkers

- In 1959, Arthur Samuel at IBM creates a computer program that plays checkers at ordinary human player level. In 1961, his program could play at a high level (State champion) using minimax and alpha-beta pruning.
- Chinook, a program developed in University of Alberta defeated world champion in 1994 (12 plays, 1 win, 11 draw). The program
  - Uses alpha-beta pruning.
  - Has a database of millions of end games.
  - Also has a database of openings.
  - Uses heuristics and knowledge about the game.

Chess

- In 1997, Deep Blue defeated world champion, Garry Kasparov.
- Current systems use parallel search, alpha-beta pruning, databases of openings and heuristics.
- The deeper in a search tree the computer can search, the better it plays.

Go

- Go is a complex game played on a 19x19 board.
- Average branching factor in search tree around 360 (compared to 38 for chess, ??? For Chinese chess).
- Google AlphaZero beat human world champion in 2016

Opening cases are very unique/important.

Computer games APP collected all world-champion games/match opening cases

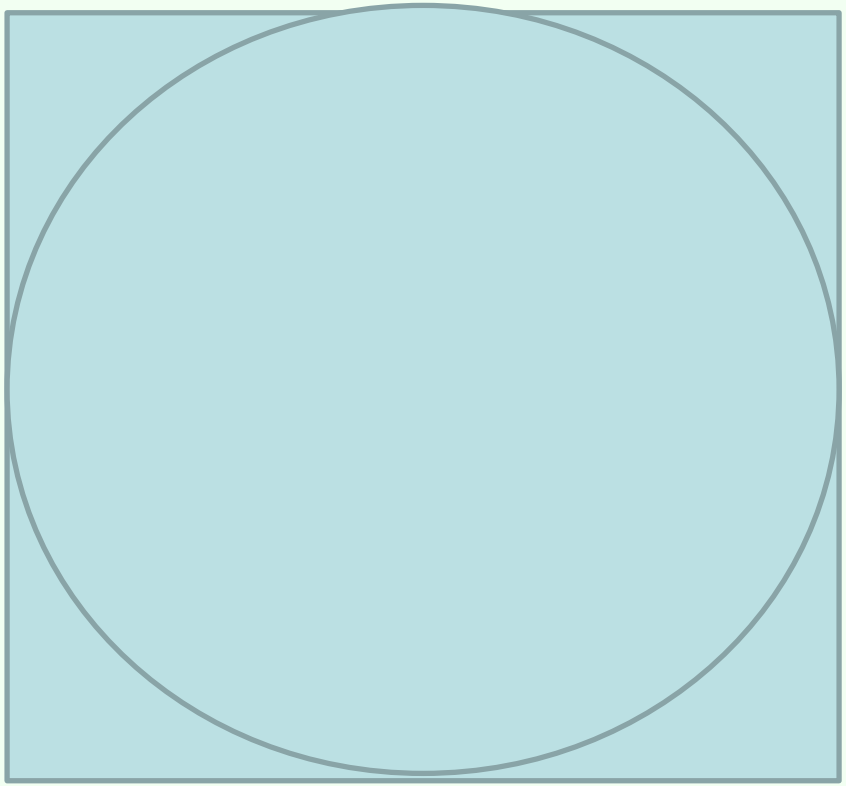
Winning/finale cases are very important and unique.

Computer games APP collected all world-champion games/match winning cases

Monte Carlo algorithm, random sampling algorithm

Example: we want to compute the value of pi = 3.14159…

We generate 1000 random 2D points (x,y), x in (-1,1), y in (-1,1), and count how many points follow inside the circle. The ratio is the value of pi.



Monte Carlo Tree Search (MCTS)

Key Concepts:

- **Monte Carlo Methods**: Use random sampling to estimate outcomes or probabilities.
- **Tree Search**: Explores a game tree by simulating possible moves and their consequences.
- **Balance Between Exploration and Exploitation**: MCTS intelligently allocates computational resources to promising branches while still exploring less-known paths.

How MCTS Works (4 Main Steps):

- **Selection**: Traverse the tree from root to a leaf node using a strategy (e.g., UCB — Upper Confidence Bound) to balance exploration and exploitation.
- **Expansion**: Add one or more child nodes to the selected leaf node (if it's not terminal).
- **Simulation**: Perform a random playout (rollout) from the new node to a terminal state.
- **Backpropagation**: Update statistics (like win rates) of all nodes along the path from the leaf back to the root.

Monte Carlo Tree Search (MCTS)

Applications:

- **Game AI**: Used in Go (AlphaGo), Chess, Shogi, and other board games.
- **Robotics**: Path planning and decision-making under uncertainty.
- **Optimization Problems**: Resource allocation, scheduling, etc.

Advantages:

Doesn’t require an evaluation function — learns from simulations.
Can handle large or infinite state spaces.
Asymmetric tree growth — focuses on promising paths.

Disadvantages:

Computationally intensive for deep or complex games.
May require many simulations for high accuracy.

AlphaGo

Developed by DeepMind (a subsidiary of Alphabet/Google)

March 2016**: AlphaGo defeated** Lee Sedol**, a 9-dan Go player, one of the world's top players by 4–1**

AlphaGo combined Monte Carlo Tree Search (MCTS) with deep neural networks:

Policy Network:

Predicts the best moves (probabilities) based on the current board state.
Trained initially on human expert games, then improved via reinforcement learning.

Value Network:

Estimates the likelihood of winning from a given board position.
Reduces the need for deep tree searches by evaluating positions directly.

Monte Carlo Tree Search (MCTS):

Uses the policy and value networks to guide simulations and select the most promising moves.

Reinforcement Learning:

AlphaGo played millions of games against itself to improve its strategy beyond human knowledge.

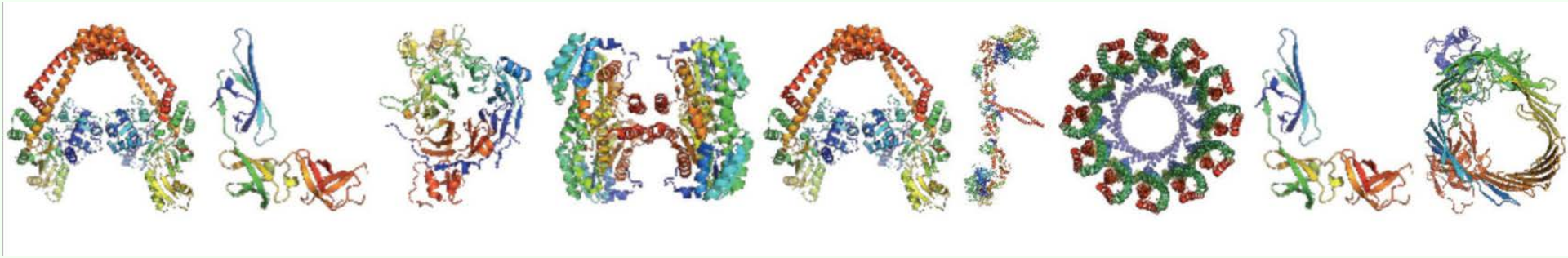
Evolution: AlphaGo → AlphaGo Zero → AlphaZero

AlphaGo Zero (2017): Learned entirely from self-play, without any human data. Surpassed all previous versions.

AlphaZero (2017): Generalized the approach to master chess and shogi as well — proving the algorithm's versatility.

Impact:

- Revolutionized AI: Showed that AI could master complex, intuitive domains previously thought to require human-like intuition.
- Inspired new research: Led to advances in reinforcement learning, self-play, and neural network architectures.
- Cultural phenomenon: Sparked global interest in AI and Go, and raised philosophical questions about human vs. machine intelligence.



AlphaFold used for predict molecular structure of Protins

Developed from AlphaZero

One chemist, two computer scientists won 2024Nobel Chemistry Prize